

# Session 2: Memory Hierarchy and Fast SIMT GEMM



周宇轩 • 林若瑜 2026-07 • Weiming HPC Training Camp × LCPU AI

Infra Seminars

## SECTION

# Part 1 — saxpy 与硬件基础

## OBJECTIVE

用一个最简单的 memory-bound kernel，建立“在飞字节”这一定量模型，并按需打开硬件。

## LISTEN FOR

一个 kernel 最快能跑多快？谁决定了这个上限？

# 一个 kernel 最快能跑多快

## 一个 kernel 最快能跑多快？

写下 kernel 之前，先问它的天花板在哪里 —— 以及天花板由谁决定

### 算

**峰值算力**：每秒能做多少 FLOP。  
受限于执行单元数量与频率。

### 搬

**峰值带宽**：每秒能从 DRAM 搬多少字节。绝大多数 kernel 卡在这里。

### 等

**延迟**：数据从发出请求到返回要多久。必须用足够多的并行工作把它盖住。

### 先看前两个

“算”和“搬”的比值决定了 kernel 落在哪一侧 —— 这就是 **Roofline**。第三个因素（延迟）Roofline 不管，我们放到后面再谈。

## Roofline 只回答 “在哪一边”

$$I = \frac{\text{浮点运算次数}}{\text{DRAM 访问字节数}} \quad [\text{FLOP/Byte}]$$

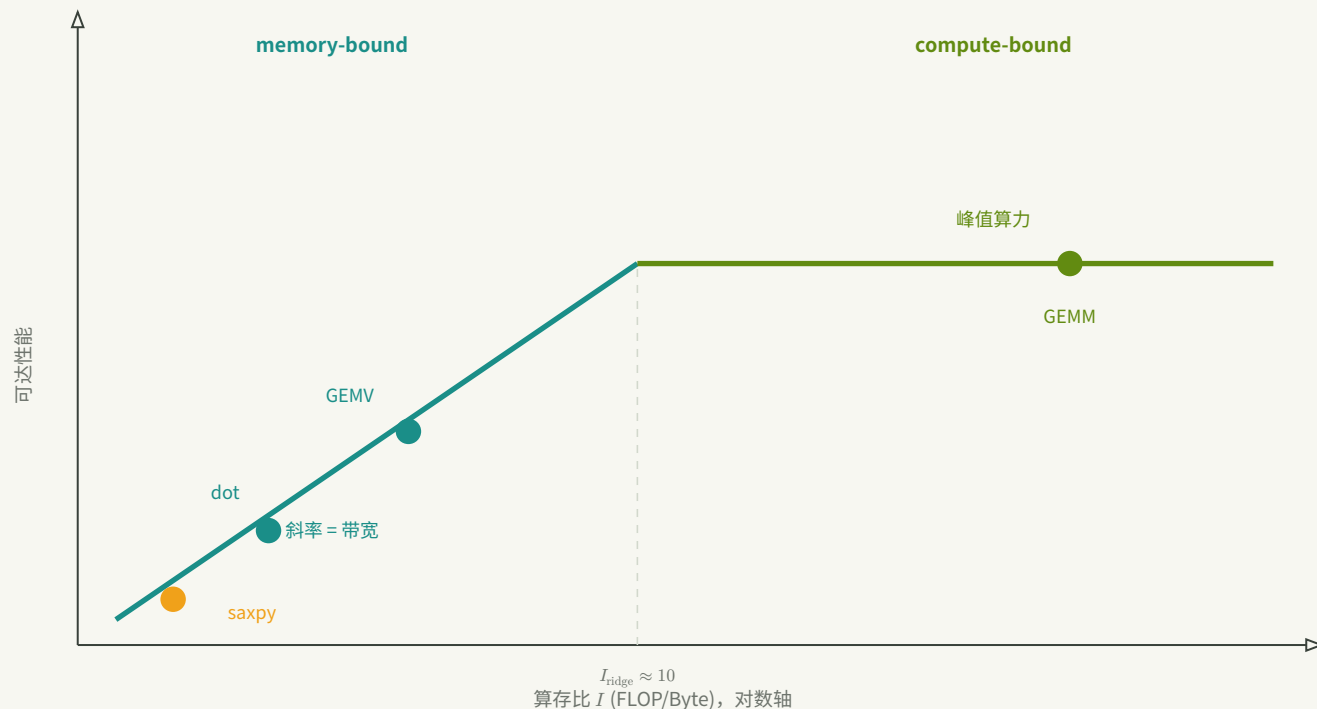
脊点  $I_{\text{ridge}} = \text{峰值算力} / \text{峰值带宽}$ ; A100 FP32 约 19.5 TFLOPS / 1.94 TB/s  $\approx$  **10 FLOP/B**

architecture-scoped

| 算子                                 | FLOP     | Bytes        | $I$                           |
|------------------------------------|----------|--------------|-------------------------------|
| saxpy $y = ax + y$                 | $2n$     | $12n$        | <b>0.17</b> — 深度 memory-bound |
| 向量点积 $x \cdot y$                   | $2n$     | $8n$         | 0.25                          |
| ReLU / LayerNorm 等 elementwise     | $\sim n$ | $8n$         | 0.1                           |
| GEMV $y = Ax$                      | $2n^2$   | $4n^2$       | 0.5                           |
| GEMM $C = AB$ ( $N \times N$ , 分块) | $2N^3$   | $\sim 12N^2$ | $\sim N/6$ — 可以 compute-bound |

GEMM 是唯一靠增大分块就能把  $I$  推过脊点的常见算子；其余绝大多数算子天生在左边 —— 这正是 Act I 的出发点

# Roofline 图



## 01 / 上限

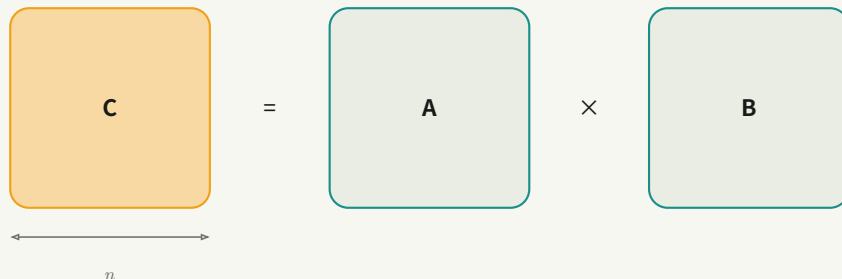
Roofline 告诉我们 saxpy 的上限是带宽。

### 但它没有告诉我们

一个完美 coalesced 的 memory-bound kernel, 到底能拿到这个上限的百分之多少?

roofline 假设 “只要在左边, 就能跑满带宽” —— 下面这个例子说明这个假设是错的。

# 大 GEMM: $I$ 随规模线性增长



数据  $\sim 3n^2$  计算  $\sim 2n^3$

计算是立方的，数据是平方的

$\rightarrow n$  越大， $I$  越高，越 compute-bound

$$\text{FLOP} = 2MNK$$

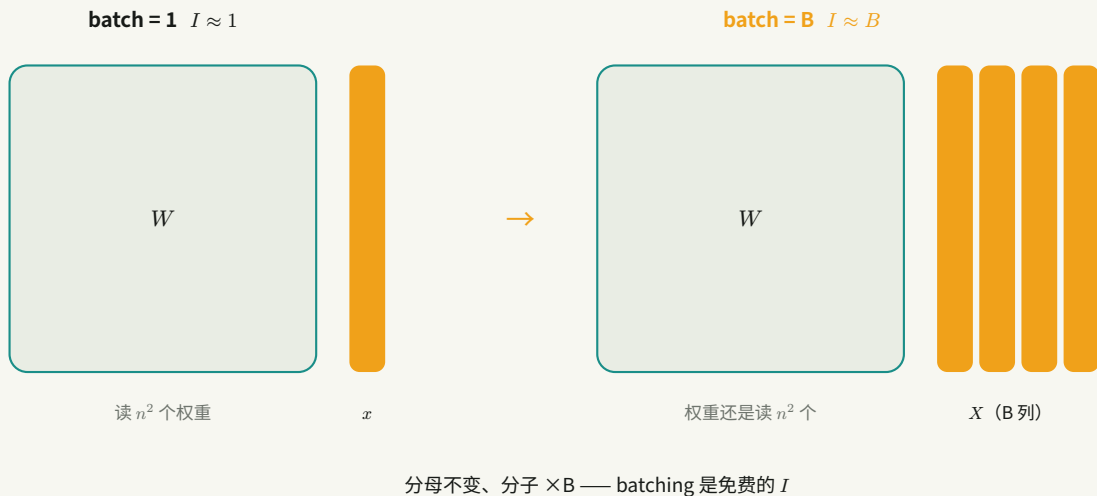
$$\text{Bytes} = 2(MK + KN + MN) \text{ (fp16)}$$

$$\text{方阵 } M = N = K = n \text{ 时: } I = \frac{2n^3}{6n^2} = n/3$$

| $n$  | $I$         | 相对 H100 TC 脊点 295 |
|------|-------------|-------------------|
| 128  | 43          | memory-bound      |
| 1024 | 341         | 刚过脊点              |
| 8192 | <b>2731</b> | <b>远在右侧</b>       |

这是理想  $I$ ，前提是每个元素只从 DRAM 读一次 —— 而这正是 Part 2 里 tiling 要做到的事。

# Decode GEMV: LLM 推理慢的根本原因



batch = 1 时每一层就是**矩阵  $\times$  向量**：  
权重  $W$  ( $n \times n$ ) 乘激活  $x$  ( $n \times 1$ )。  
 $\text{FLOP} = 2n^2$ ,  $\text{Bytes} \approx 2n^2 \Rightarrow I \approx 1$

**每个权重从 DRAM 读进来，只参与一次乘加就被丢掉。**

## 70B fp16 模型，H100

|                                              |                                           |
|----------------------------------------------|-------------------------------------------|
| 每 token 需搬权重                                 | 140 GB                                    |
| HBM3 带宽                                      | 3.35 TB/s                                 |
| <b>带宽下限 <math>\rightarrow</math> 每 token</b> | <b><math>\approx 42 \text{ ms}</math></b> |
| 对应上限                                         | $\approx 24 \text{ tok/s}$                |

**权重量化**（缩小分母）与**投机解码**（等价于提高 batch）之所以有效，都是同一件事：在 roofline 上往右、往上挪。

# axpy: 一个再简单不过的 kernel

```
1 template < typename scalar_t >
2 __global__ void axpy_naive(scalar_t a, const scalar_t * x, scalar_t * y, int n) {
3     int tid = threadIdx.x + blockIdx.x * blockDim.x;
4     if (tid < n) {
5         y[tid] = a * x[tid] + y[tid];
6     }
7 }
```

CUDA

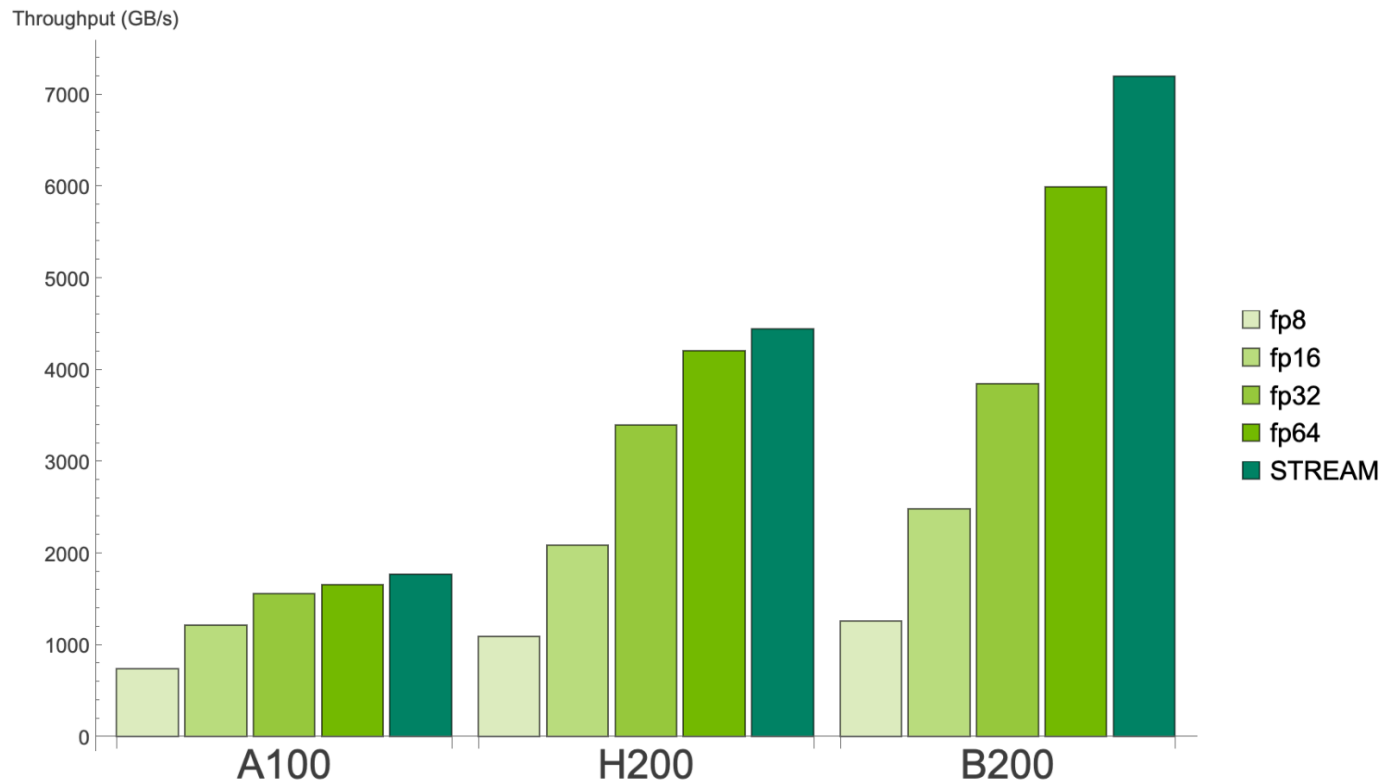
## 挑不出毛病

- 完美 **coalesced**: 相邻线程访问相邻地址
- 零分支、零同步、零 shared memory
- 100% **occupancy** (只用十几个寄存器)
- embarrassingly parallel, 无数据依赖

模板参数 `scalar_t` 让同一份代码换不同 datatype 编译 —— 下一页就用它做实验。



## Part 1 — saxpy 与硬件基础



### 01 / 两个问题

代码一个字没改，只换了

```
scalar_t o
```

#### ① 为什么窄类型更慢？

fp8 / fp16 搬的字节更少，  
却离 STREAM **更远**。

#### ② 为什么卡越新越明显？

A100 上 fp32 / fp64 都**接近饱和**； B200 上同样的  
代码却明显掉队。

# 把差距量化

每格为 achieved / STREAM

A100 measured

course design

| GPU  | fp8 | fp16 | fp32 | fp64 |
|------|-----|------|------|------|
| A100 | 41% | 69%  | 88%  | 94%  |
| H200 | 25% | 47%  | 77%  | 95%  |
| B200 | 18% | 34%  | 53%  | 83%  |

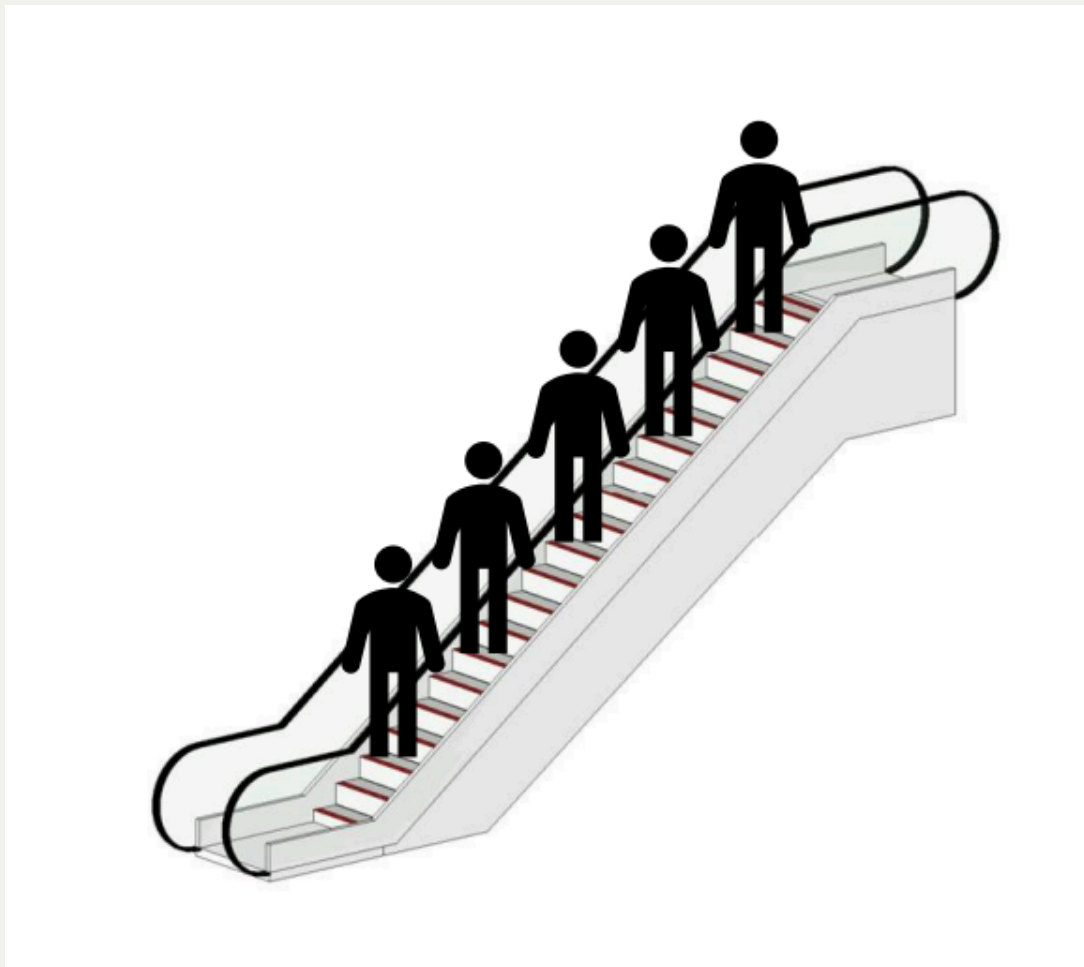
沿每一行往左走，缺口变大；沿每一列往下走，缺口也变大。两个方向共同点是：每个线程一次搬的字节太少。

## 同一个解释

fp8 每线程只搬 1 字节，fp64 搬 8 字节 —— 差 8 倍。而 B200 的带宽是 A100 的 4 倍，要填满它需要的在途数据也成倍增加。Roofline 里没有任何一项能表达这件事。

缺的这个量叫在飞字节 (bytes in flight) —— 下面先用一部扶梯把它讲清楚。

## 先看一部扶梯



### 扶梯参数

- 一级台阶站 **1 个人**
- 每 **2 秒** 来一级台阶 → **带宽** = 0.5 人/秒
- 梯身 **20 级** → **延迟** = 40 秒

要占满这部扶梯，需要多少人同时在梯上？

$$\begin{aligned}\text{并发量} &= \text{带宽} \times \text{延迟} \\ &= 0.5 \text{ 人/秒} \times 40 \text{ 秒} \\ &= \mathbf{20 \text{ 人}}\end{aligned}$$

少于 20 人，梯上就有空台阶。

# Little' s Law: 带宽不是免费的

在飞字节 = 带宽  $\times$  延迟

把扶梯上的“人”换成“字节”—— Little' s Law ( $L = \lambda W$ ) 在内存系统上的形式 course design

## 含义

想让内存总线以带宽  $B$  持续吐数据，就必须**始终**有  $B \times L$  字节的请求**已经发出、但还没返回**。凑不够这个数，总线就有空档。

## 代入 A100

$1935 \text{ GB/s} \times 500 \text{ ns} \approx \mathbf{1 \text{ MB}}$

必须同时在途

H100:  $3.35 \text{ TB/s} \times 600 \text{ ns} \approx 2 \text{ MB}$ ; **越新的卡这个数越大**

## 回答开头的问题②

延迟没怎么变，带宽却翻了几倍——所以**越新的卡，要填满它需要的在飞字节越多**。同一份代码在 B200 上离饱和更远，不是代码变差了，是门槛变高了。

## 摊到每个线程头上

A100 满占用：108 SM × 2048 线程 = **22 万** 个线程

$$\text{每线程需要的在飞字节} = \frac{1 \text{ MB}}{22 \text{ 万}} \approx 4.7 \text{ B}$$

回头看朴素 saxpy：每个线程发 **2 条 4 B load**（读  $x$ 、读  $y$ ）= 8 B 在飞。

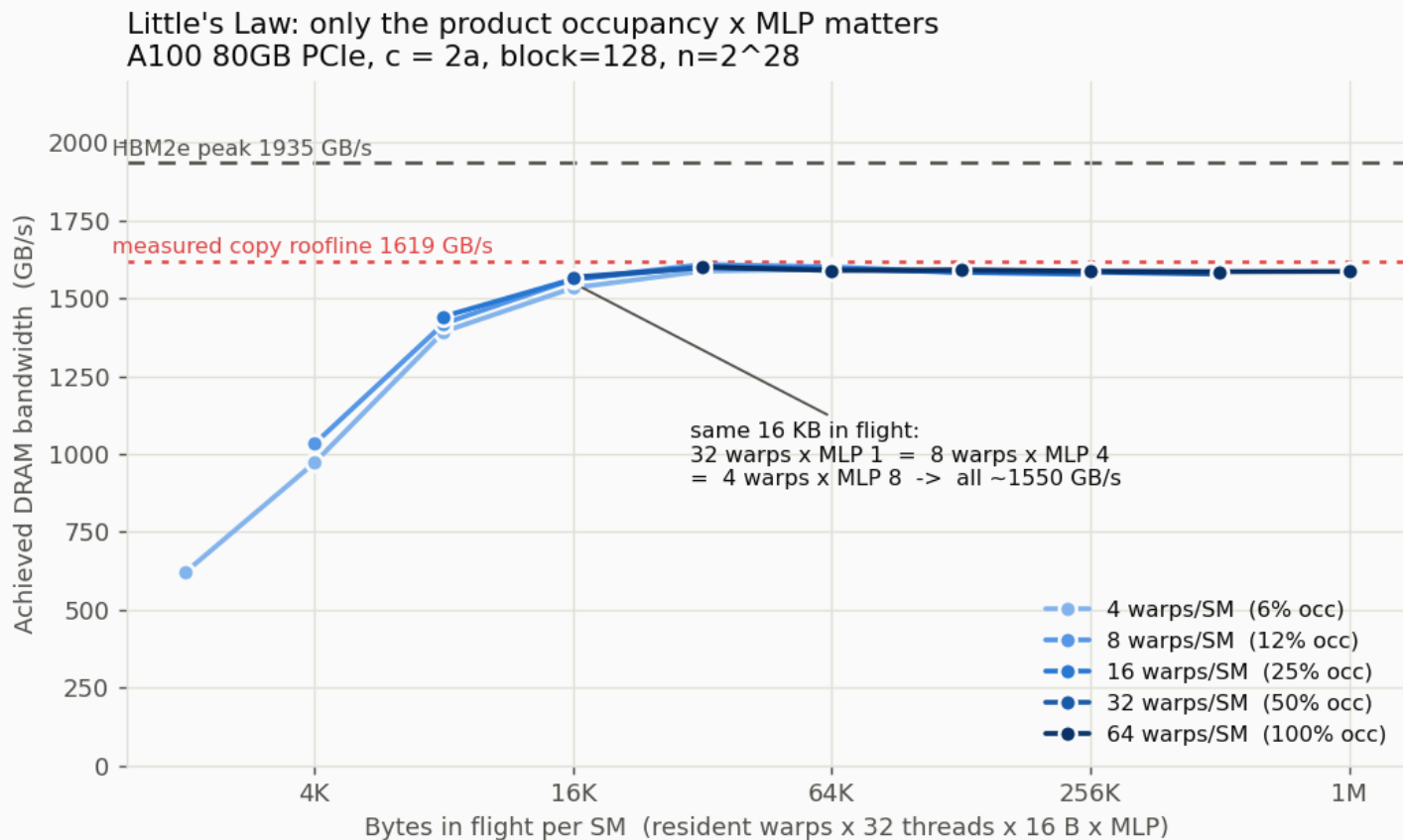
### 每 SM 在飞

|                      |          |               |
|----------------------|----------|---------------|
| 朴素 saxpy（满占用，8 B/线程） | 16 KB    | 勉强够           |
| Little's Law 膝点（实测）  | 16~32 KB | <b>正好卡在边缘</b> |

### 回答开头的问题①

8 B/线程 × 满占用 = 16 KB/SM，**正好落在膝点上**，实测 82%（曲线预测 80%）。**换成 fp16 就只剩 4 B/线程**，掉到膝点左边 —— 这就是窄类型变慢的原因。

## Part 1 — saxpy 与硬件基础



### 01 / 扫描

固定  $c = 2a$ ，二维扫描「每 SM 驻留 block 数」×「每线程在飞 load 数 (MLP)」。

横轴 = 每 SM 在飞字节 = 驻留 warp 数 ×  $32 \times 16 \text{ B} \times \text{MLP}$ 。

### 结论

30 个配置，一条曲线

A100 measured

## 这张图说了什么

| 每 SM 在飞 | 配置              | GB/s |
|---------|-----------------|------|
| 16 KB   | 32 warp × MLP 1 | 1568 |
| 16 KB   | 16 warp × MLP 2 | 1562 |
| 16 KB   | 8 warp × MLP 4  | 1561 |
| 16 KB   | 4 warp × MLP 8  | 1534 |

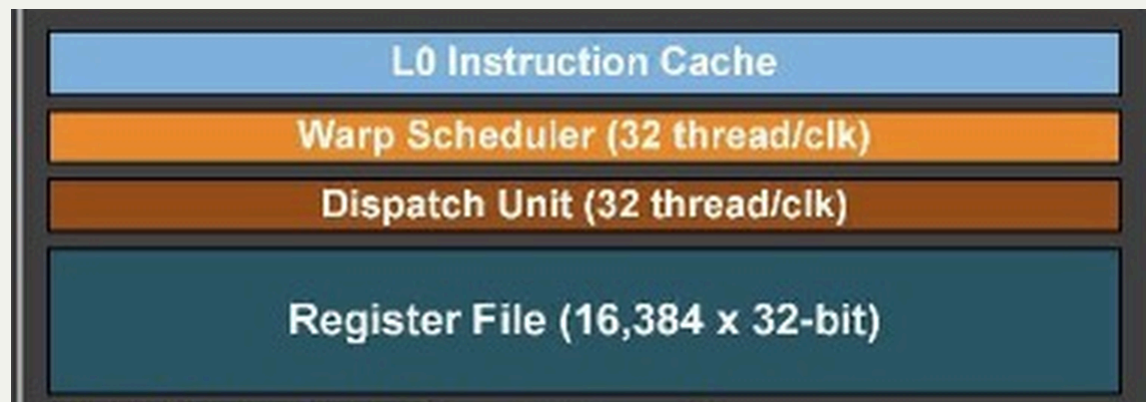
四个配置的 occupancy 差 8 倍、MLP 差 8 倍，带宽差 2%。

于是提高带宽只有两条路

硬件只关心乘积，不关心你是靠哪个因子凑出来的：更多的 warp（occupancy），或者每个 warp 发更多 / 更宽的访存（ILP / 向量化）。

## 路线一: 为什么 occupancy 能变成带宽

一个 SM 被切成 4 个 sub-partition，每个都是一台相对独立的调度机器：



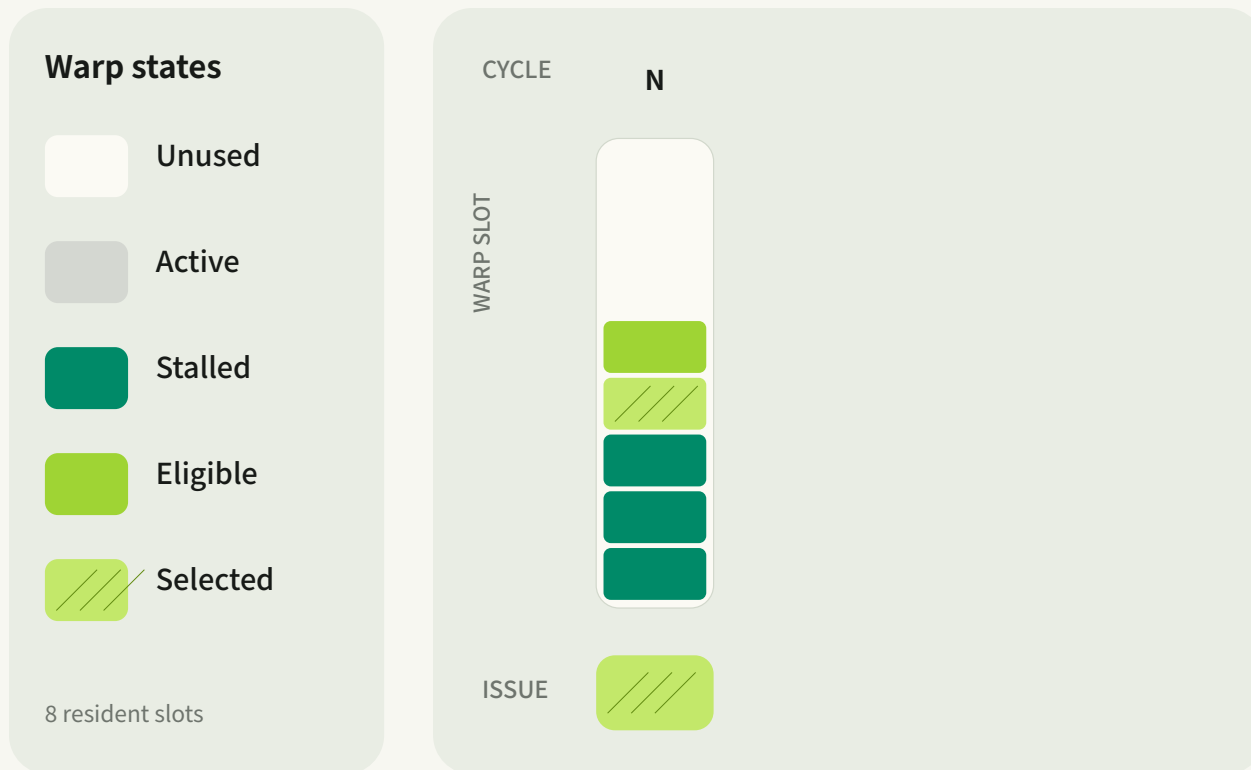
- 自己的 **warp scheduler** + dispatch unit
- 自己的 **16384 × 32-bit 寄存器堆**
- 一个 SM 共 **64 K** 寄存器、最多驻留 **64 warp**

底部的 256 KB L1 / shared memory 是 4 个 partition 共享的。图为 Hopper；A100 的四分区结构相同，但 L1/shared 容量为 192 KB。

architecture-scoped

Hopper SM • 引自 NVIDIA Hopper 架构白皮书





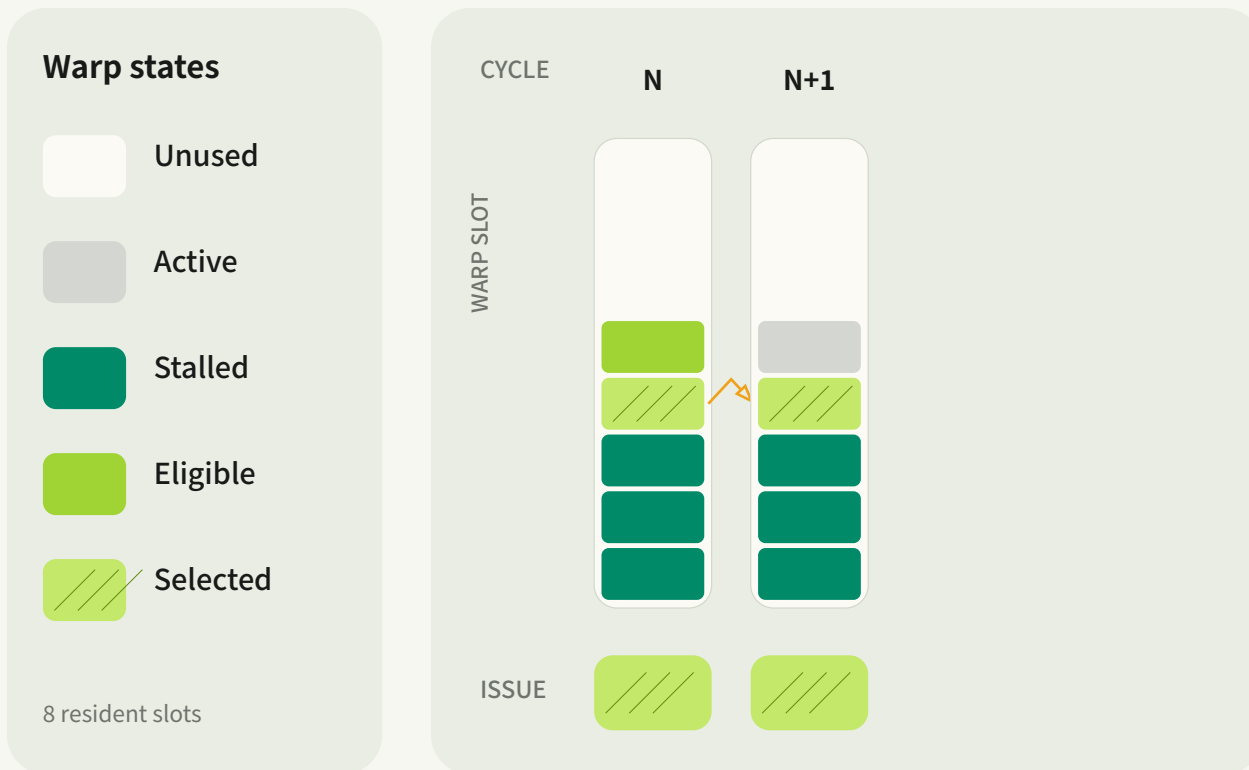
## 01 / N

scheduler 从 eligible 的 warp 里选中一个并**发射**。

### 0 cycle 切换

每个 scheduler 手上驻留多个 warp slot, **每个 cycle** 重新选一个。切换 warp **不需要保存/恢复现场** —— 每个 warp 的寄存器**一直在寄存器堆里**。

CPU 换线程要几百到几千 cycle; GPU 换 warp 是 **0 cycle**。



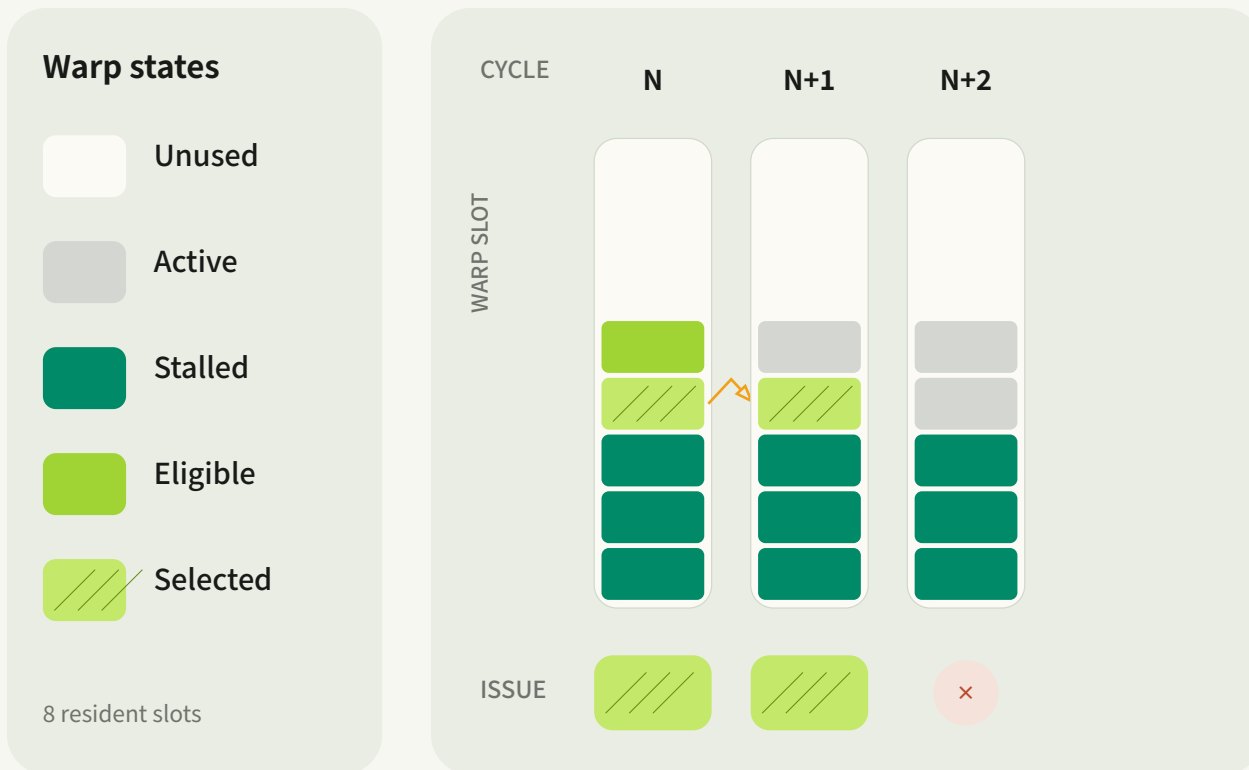
02 / N+1

scheduler 从 eligible 的 warp 里选中一个并**发射**。

### 0 cycle 切换

每个 scheduler 手上驻留多个 warp slot, 每个 **cycle** 重新选一个。切换 warp **不需要保存/恢复现场** —— 每个 warp 的寄存器**一直在寄存器堆里**。

CPU 换线程要几百到几千 cycle; GPU 换 warp 是 **0 cycle**。



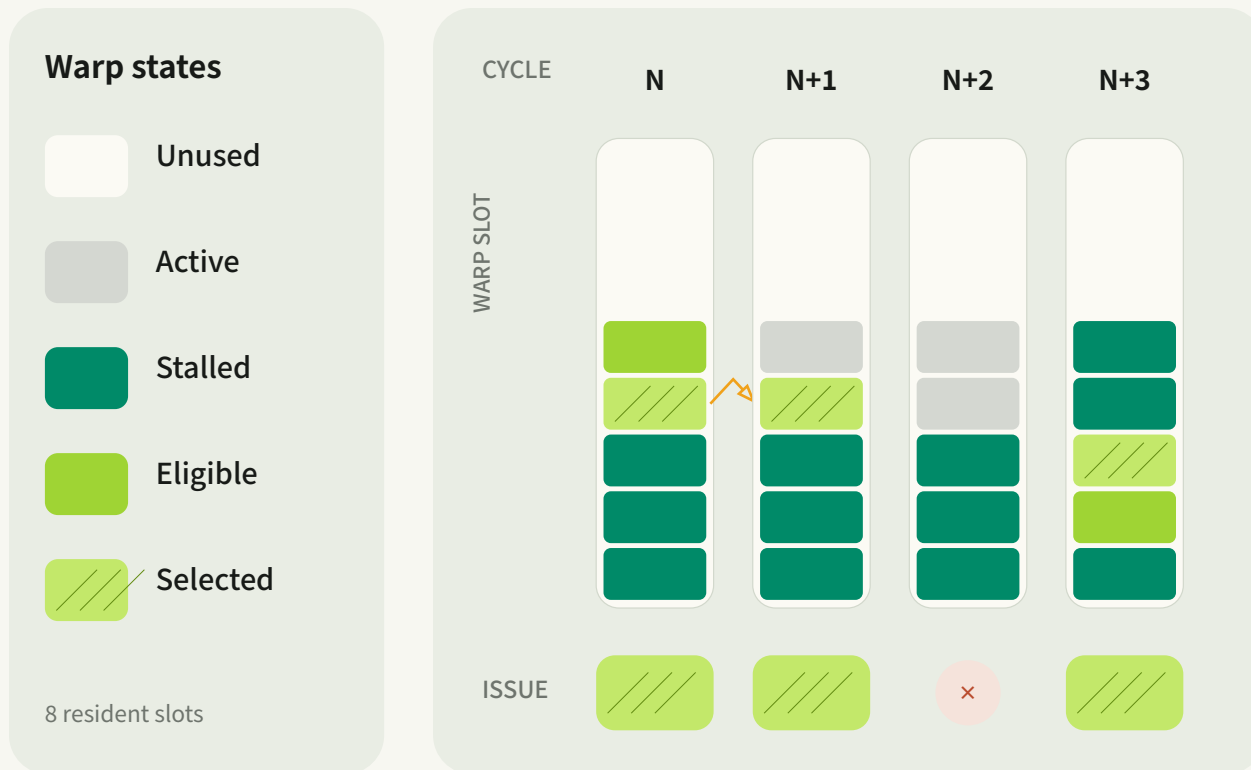
### 03 / N+2

没有 eligible warp，这个 cycle 的发射槽**空置**。

#### 0 cycle 切换

每个 scheduler 手上驻留多个 warp slot，**每个 cycle** 重新选一个。切换 warp **不需要保存/恢复现场** —— 每个 warp 的寄存器**一直在寄存器堆里**。

CPU 换线程要几百到几千 cycle；GPU 换 warp 是 **0 cycle**。



## 04 / N+3

scheduler 从 eligible 的 warp 里选中一个并**发射**。

### 0 cycle 切换

每个 scheduler 手上驻留多个 warp slot, **每个 cycle** 重新选一个。切换 warp **不需要保存/恢复现场** —— 每个 warp 的寄存器**一直在寄存器堆里**。

CPU 换线程要几百到几千 cycle; GPU 换 warp 是 **0 cycle**。

## 于是访存指令可以连续发射



- warp A 发完 load 就 stall → scheduler 立刻从 warp B 发下一条 load
- 每条 load 都在**总线上叠加**，在飞字节线性增长
- 直到凑够 带宽 × 延迟，总线才真正被填满

注意**不是**“更多 warp 算得更快”—— 计算根本不是瓶颈 (`issue_active` 实测只有 1.5%~4.3%)。是**更多 warp 能同时挂起更多访存请求**。

## 实测: 占用扫描

`c = 2a`, 固定 MLP=1、标量访存, 只改每 SM 驻留的 warp 数 A100 measured

| 每 SM warp 数 | occupancy | 每 SM 在飞 | GB/s |
|-------------|-----------|---------|------|
| 4           | 6.25%     | 2 KB    | 620  |
| 8           | 12.5%     | 4 KB    | 1033 |
| 16          | 25%       | 8 KB    | 1442 |
| 32          | 50%       | 16 KB   | 1568 |
| 64          | 100%      | 32 KB   | 1602 |

从 6.25% 到 100%, 带宽 **2.6 倍**。这是所有因素里**唯一可能造成数量级差距**的。

### 推论

**grid 一定要够大**。只 launch 108 个 block (每 SM 恰好 1 个) 会比满 grid 慢 23%~61%。

## 路线二: 提高单指令的字节数

$$\text{每 SM 在飞字节} = \underbrace{\text{驻留 warp 数}}_{\text{occupancy}} \times 32 \times \underbrace{\text{单指令字节数}}_{\text{向量化}} \times \underbrace{\text{“MLP”}}_{\text{展开}}$$

occupancy 有硬上限 (64 warp)，而且常常被寄存器 / smem 压着。那就从**第二个因子**下手 —— 让一条指令搬更多字节。

| 类型     | 位宽      | SASS      | 一个 warp 的请求            |
|--------|---------|-----------|------------------------|
| float  | 32 bit  | LDG.E     | 128 B = 1 条 cache line |
| float2 | 64 bit  | LDG.E.64  | 256 B                  |
| float4 | 128 bit | LDG.E.128 | 512 B = 4 条 cache line |

# 标量 vs 向量化

## 标量

```
int i = blockIdx.x * blockDim.x
    + threadIdx.x;
if (i < n)
    y[i] = a * x[i] + y[i];
```

CUDA

- 每线程 2 条 4 B load
- 在飞 8 B

## 向量化

```
int i = blockIdx.x * blockDim.x
    + threadIdx.x;
if (i < n/4) {
    float4 vx = ((float4*)x)[i];
    float4 vy = ((float4*)y)[i];
    vy.x = a * vx.x + vy.x; // ...
    ((float4*)y)[i] = vy;
}
```

CUDA

- 每线程 2 条 16 B load
- 在飞 32 B



# 实测：向量宽度

## 写法

|                       | 单指令字节 | GB/s        | 占峰值          |
|-----------------------|-------|-------------|--------------|
| <code>float</code> 标量 | 4 B   | 1498        | 77.4%        |
| <code>float2</code>   | 8 B   | 1669        | 86.3%        |
| <code>float4</code>   | 16 B  | <b>1685</b> | <b>87.1%</b> |

fp32 上 `float4` 相对标量 **+12%** —— 真实，但不是数量级差别。窄数据类型下差距才真正拉开：

## 写法

|                                          | 每线程元素 | GB/s        | 占峰值          |
|------------------------------------------|-------|-------------|--------------|
| <code>__half</code> 一元素一线程 (16-bit load) | 1     | <b>1112</b> | 57.5%        |
| <code>__half2</code> (32-bit)            | 2     | 1427        | 73.8%        |
| <code>half8</code> (128-bit)             | 8     | <b>1592</b> | <b>82.3%</b> |

标量 fp16 只有 2 B/线程在飞，满占用也只有 8 KB/SM —— **膝点左边**

## 两条路是可以互相替代的

低占用时，靠每线程多发几条 load (ILP) 几乎可以**完全补偿**：

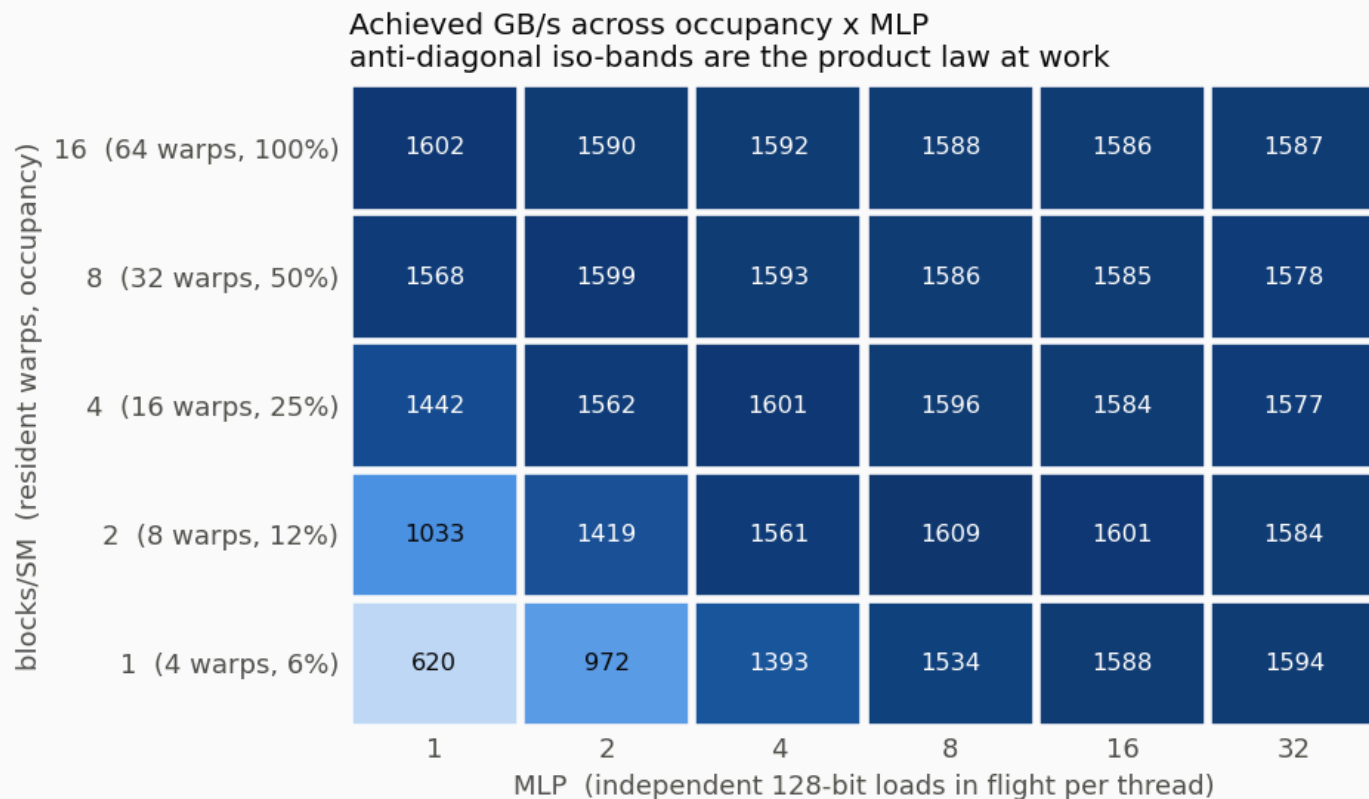
| 4 warp/SM (6.25% 占用) | 每 SM 在飞 | GB/s        |
|----------------------|---------|-------------|
| MLP = 1              | 2 KB    | 620         |
| MLP = 4              | 8 KB    | 1393        |
| MLP = 8              | 16 KB   | <b>1534</b> |
| 参照：满占用最好成绩           | 32 KB   | 1602        |

6.25% 占用 + MLP=8 达到 1534 GB/s，与满占用最好成绩**只差 4%**。

### 反过来也成立

**满占用时 MLP 完全无所谓**。64 warp/SM 下 MLP=1 是 1602，MLP=32 是 1587 —— 一条 128-bit load 就够了，因为已经越过膝点。

# 实测热力图



## 01 / 形状

等值带沿**反对角线**分布。

### 读法

这正是“乘积决定一切”的形状：沿反对角线移动，occupancy与MLP此消彼长，乘积不变 → 带宽不变。

# 回头看 Roofline: 它是一个一阶模型

Roofline 只有**两个参数**（峰值算力、峰值带宽），所以它只能回答一个问题：**如果唯一的瓶颈是 DRAM 带宽或算力，我最快能到多少。**

## 隐含前提

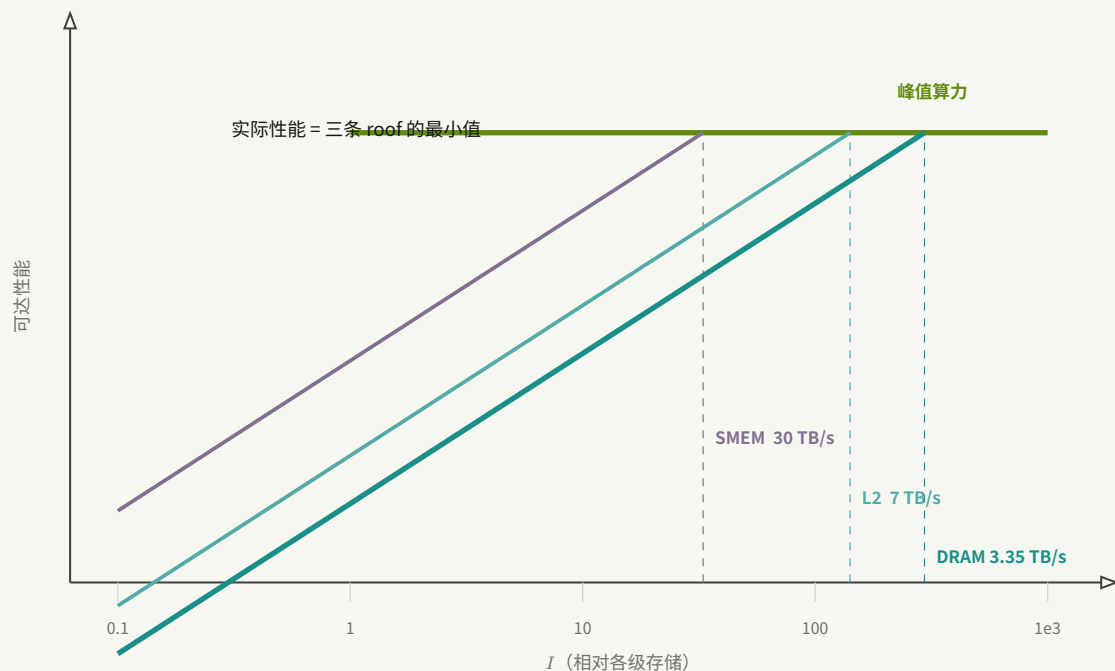
## 被打破时的真实瓶颈

|               |                                        |     |
|---------------|----------------------------------------|-----|
| DRAM 是唯一的带宽层级 | SMEM / L2 先饱和 → hierarchical roofline  | 破例一 |
| 指令只有访存和浮点     | 地址计算、循环、类型转换把 issue slot 吃光            | 破例二 |
| 延迟总能被并发掩盖     | 在飞请求不够，带宽根本没被激发出来                      | 已讲  |
| 工作量能均匀铺满整卡    | tail effect / wave quantization, SM 空转 | 破例三 |

## 不是说 roofline 没用

它是唯一能给出“还差多远”这个数的模型。但它给的是**上界**，而打不到上界的原因，都在这四条里——第三条就是刚讲完的 Little's Law。

## 破例一: hierarchical roofline



DRAM 那条斜边不是唯一的天花板。同一个 kernel 在**每一级存储**上都有自己的  $I$  (分母换成“从该级搬的字节数”), 于是每级都有自己的斜边 —— 带宽越高, 斜边越陡、脊点越靠右。

### tiling 只是把搬运推到下一级

shared-memory tiling 让 DRAM 的  $I$  变大, 但 **SMEM 的  $I$  一点没变**。所以 Part 2 里的 **register tiling** 才是必需的 —— 它提高的是相对 SMEM 的  $I$ 。

离 DRAM 峰值还远却上不去, 很可能是 **L2 / L1 先饱和** —— 看 NCU 的 SOL 谁占用最高。

## 破例二: issue bound —— 瓶颈是指令条数，不是字节数

每个 scheduler 每 cycle 只能发射 1 条指令，而地址计算、循环控制、边界判断、类型转换全都要占 issue slot。

```
1 for (int i = tid; i < n; i += stride) {  
2     int r = i / W;          // IMAD + 移位  
3     int c = i - r * W;      // IMAD  
4     out[r * W + c] = in[r * W + c] * s;  
5 }
```

CUDA

解法都是“用更少的指令搬同样多的字节”：

- **向量化** (`float4`)：字节数不变，指令数变 1/4
- `#pragma unroll`：摊薄循环的比较与自增
- **预算好索引**：不变量提到循环外，别重复整数乘除
- **Hopper TMA**：一条指令描述整个多维 tile，地址计算开销归零

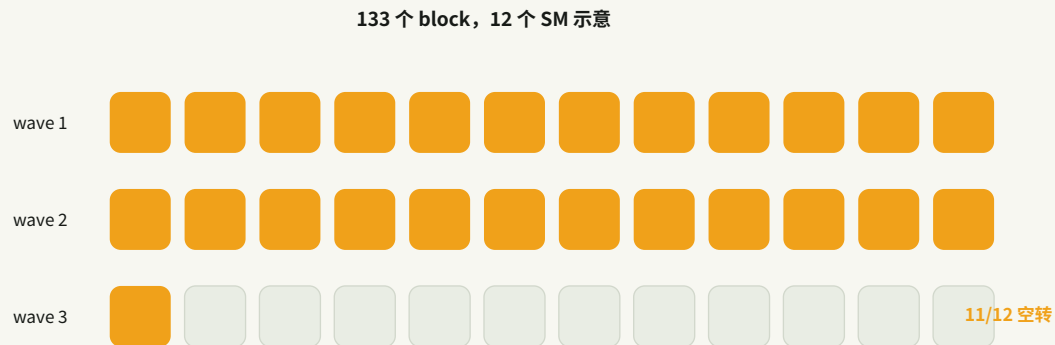
### 典型症状

NCU 里 Compute 和 Memory 两个 SOL 都不高，但

`smsp_inst_executed` 很大，warp stall 主要是 `no_instruction` / `dispatch_stall`。

回看前面 `float4` 的 +12%：一部分是在飞字节变多（路线二），另一部分是指令条数变成 1/4 —— 后者 Roofline 里根本没有对应的项。

## 破例三: tail effect / wave quantization



耗时 = 3 个 wave, 有效工作 = 2.08 个

利用率 69%

GPU 以 **wave** 为粒度铺 block: 一个 wave = 所有 SM 各驻留满一批 block。最后一个 wave 没铺满, 剩下的 SM 就**纯粹空转**, 而 kernel 必须等它跑完。

| block 数 | wave 数 | 利用率   |
|---------|--------|-------|
| 132     | 1      | 100%  |
| 133     | 2      | 50.4% |
| 264     | 2      | 100%  |
| 265     | 3      | 66.9% |
| 1320    | 10     | 100%  |
| 1321    | 11     | 91.0% |

### GEMM 里的对应现象

block 数越大量化误差越被摊薄, 这是“**grid 一定要够大**”的另一半理由。

GEMM 里的对应物叫 **tile quantization**:

8192 被 tile  $128 \times 128$  整除, 改成 8200 就多出一排几乎空转的 tile。



SECTION

# Act I — Preview

OBJECTIVE

从 GEMM 语义、data reuse 与简单 Roofline 建立完整优化地图。

---

LISTEN FOR

哪些数据值得复用？复用到哪一层？



# GEMM Workload: 计算语义与 Reuse

GEMM 语义、三个 workload-level 问题、reuse 结论。

# 第一次引入 Roofline

每搬一个 byte 只做少量计算时，性能受数据移动限制。

CeTZ Roofline: 两段屋顶 + memory-bound / compute-bound 区域。

# GEMM 的 Roofline Paradox

算法级: Bytes<sub>min</sub>  $\approx 4(MK+KN+MN)$ , AI  $\approx N/6$ 。

Naive 代码级: Bytes  $\approx 8MNK$ , AI  $\approx 0.25$  FLOP/byte。

# 用 Tiling 说明 Reuse 的数量级

CeTZ: 一个 block 的 C tile + A/B tile 装载一次、复用多次。

shared tiling 的首要价值是捕获 block-level reuse，减少高层数据移动。

# GEMM Evolution Map (V0 → V6)

CeTZ: 一条 V0→V6 的路线图（持久图，后续 Act III 逐版本回访）。

## SECTION

# Act II — Explain

## OBJECTIVE

逐层打开 GPU 硬件，用设计约束解释每项优化为什么存在。

## LISTEN FOR

为什么相邻线程最好访问相邻地址？为什么 shared memory 需要显式管理？

## Episode 1 — SIMT 与 Warp 执行

软件写 scalar code，硬件以 warp 为执行/发射单位。

CeTZ: warp = 32 scalar threads，共享 instruction issue，32 lanes。

# Episode 1 — Coalescing 与 Sector

coalescing 提高每个 request 的有效性，不等于固定 128-byte transaction。

CeTZ: 32 lanes x 4B 连续 load → 4 个对齐 32B sector; 跨 boundary 变多。



# Episode 1 — Copy 微基准与 NCU

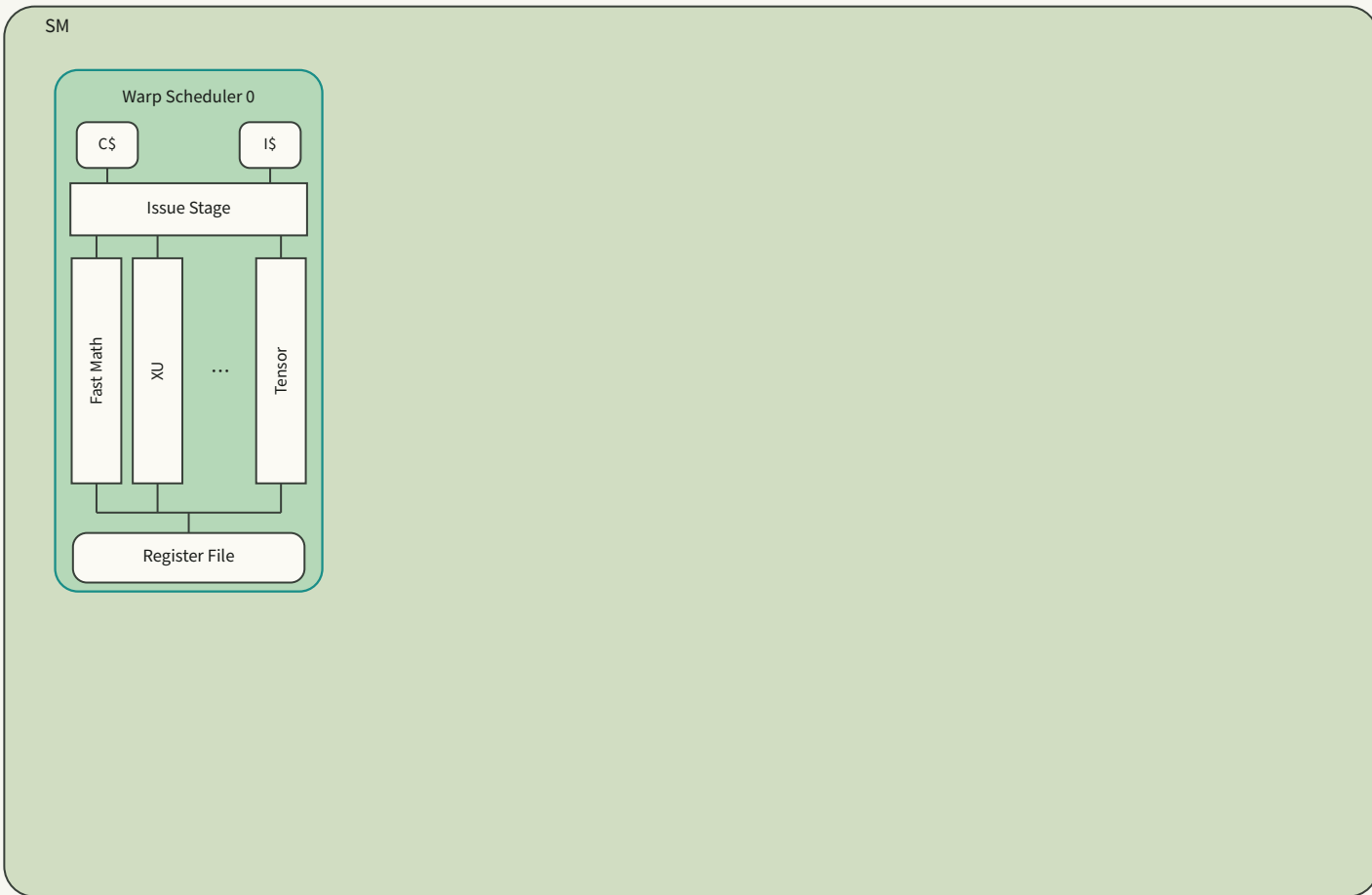
先隔离 coalescing，再回到 GEMM。

code-block: `out[i] = in[i];` 与 stride/转置式访问。

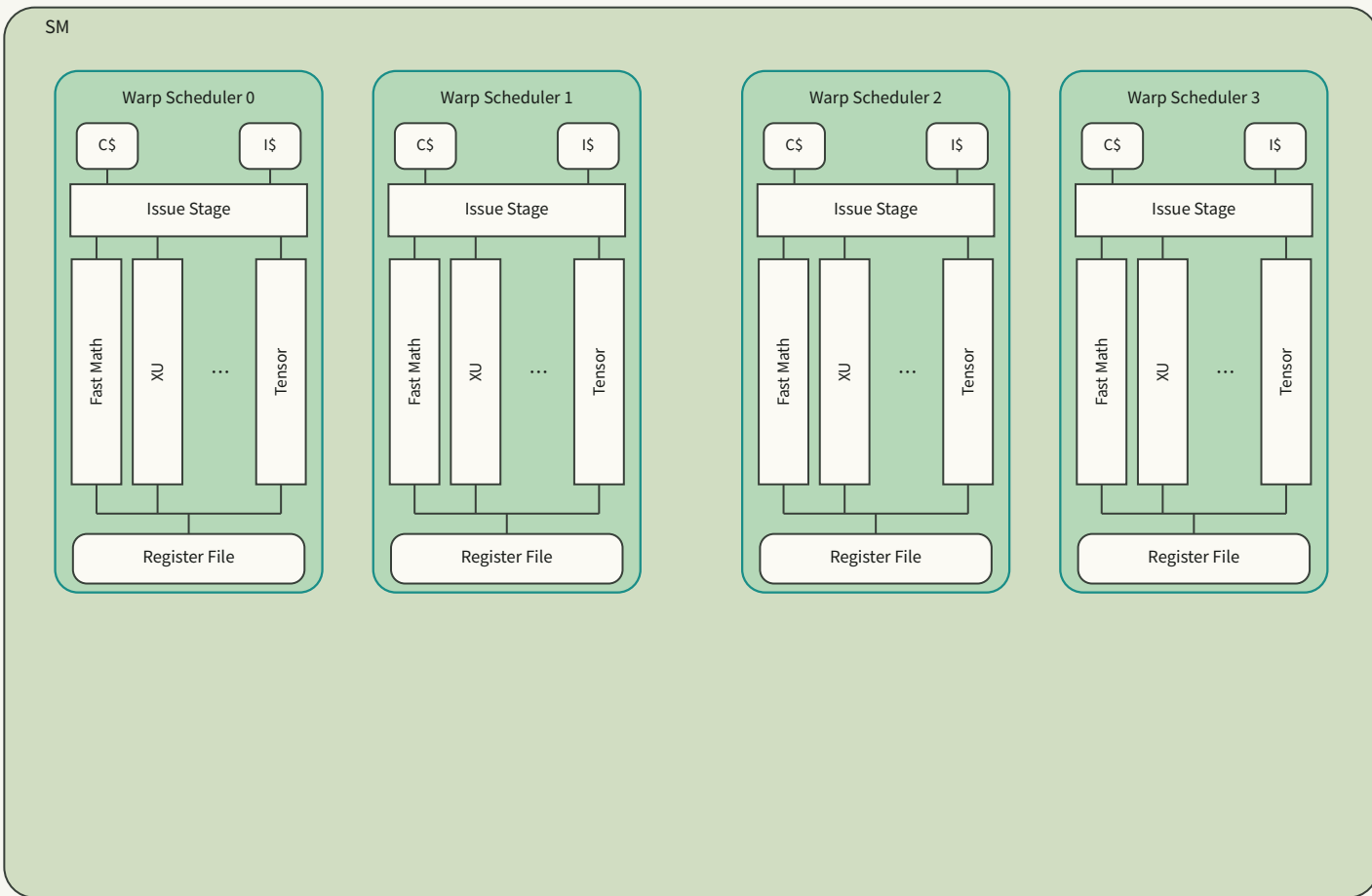
## Episode 2 — A100 SM 教学模型 (SMSP)

### 01 / Anchor

SM 外壳 + 第一个 SMSP:  
warp scheduler、register  
file、FP/INT 执行单元。



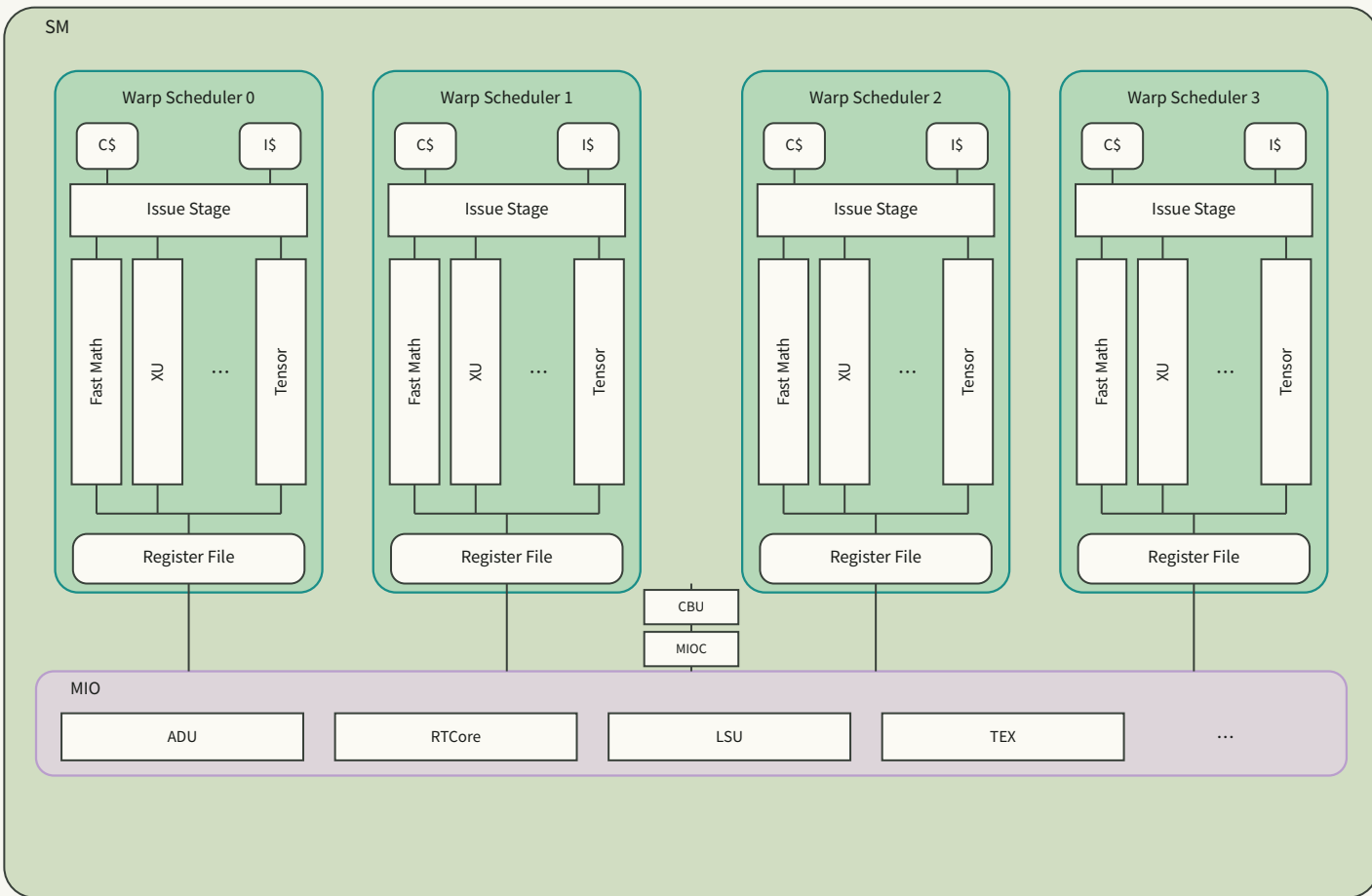
## Episode 2 — A100 SM 教学模型 (SMSP)



### 02 / Partitions

补齐 4 个 SMSP，每个都有自己的 scheduler 与寄存器堆。

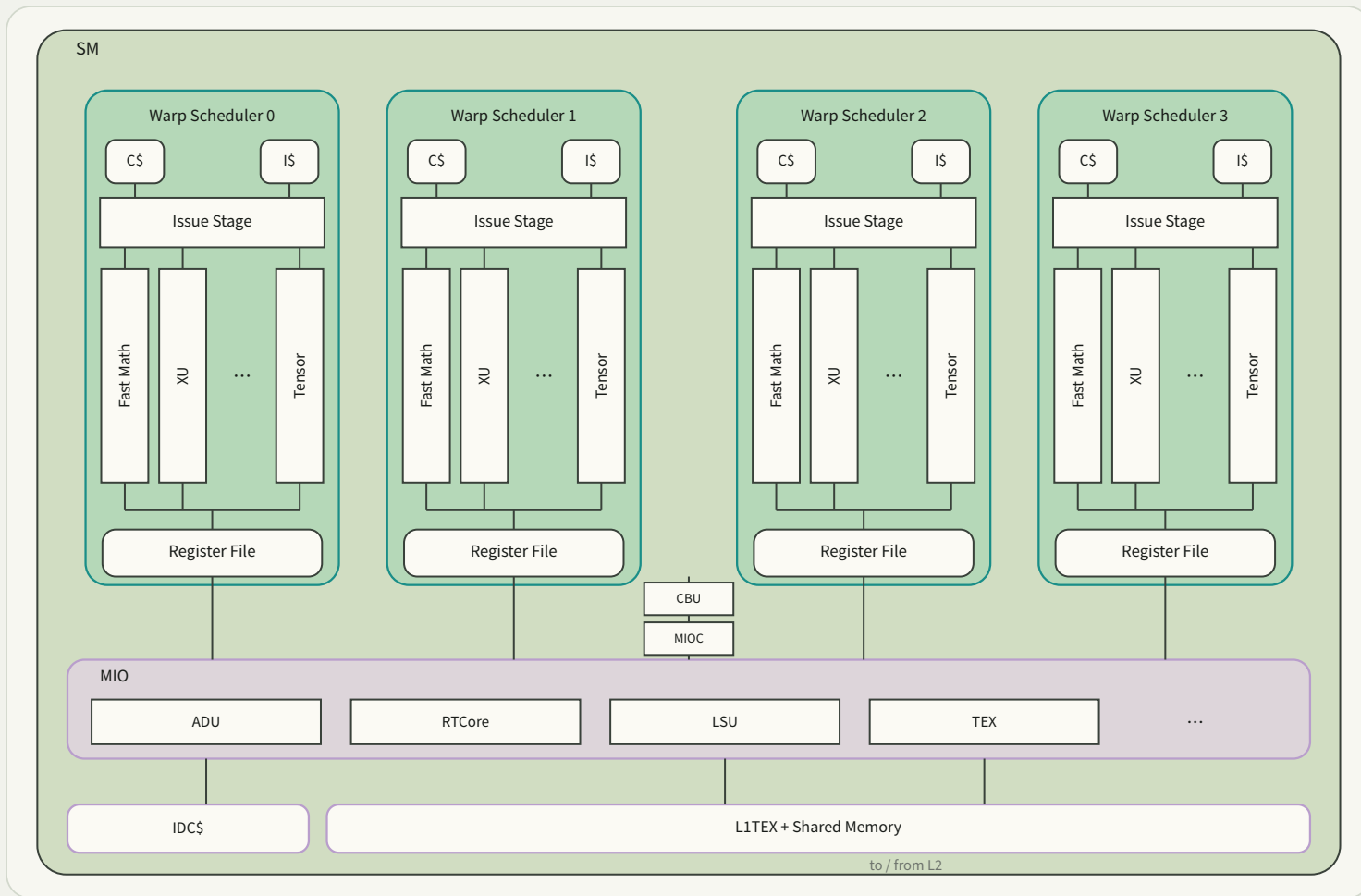
# Episode 2 — A100 SM 教学模型 (SMSP)



## 03 / MIO

执行结果经输出总线汇入  
MIO: ADU / RTCore /  
LSU / TEX。

# Episode 2 — A100 SM 教学模型 (SMSP)



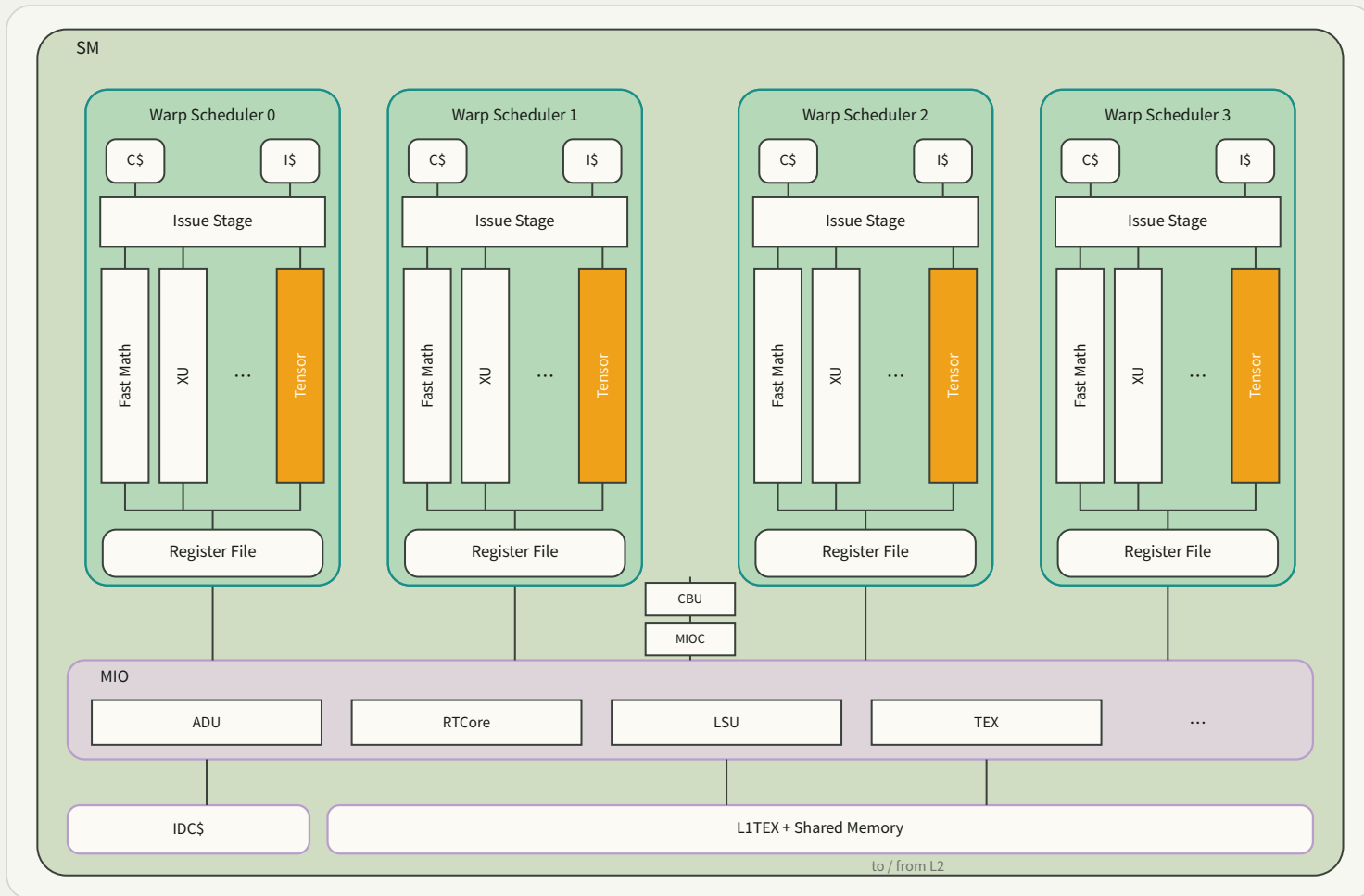
## 04 / Cache

接上 L1TEX + Shared Memory 与 IDC\$, 向下连 L2。

# Episode 2 — A100 SM 教学模型 (SMSP)

## 05 / Spotlight

语义 spotlight: 4 个  
SMSP 的 Tensor 单元高亮  
——当前关注的计算路径。



## Episode 2 — Scoreboard 与 Latency Hiding

GPU 不把 dependent load 变便宜，而是用其他独立工作覆盖延迟。

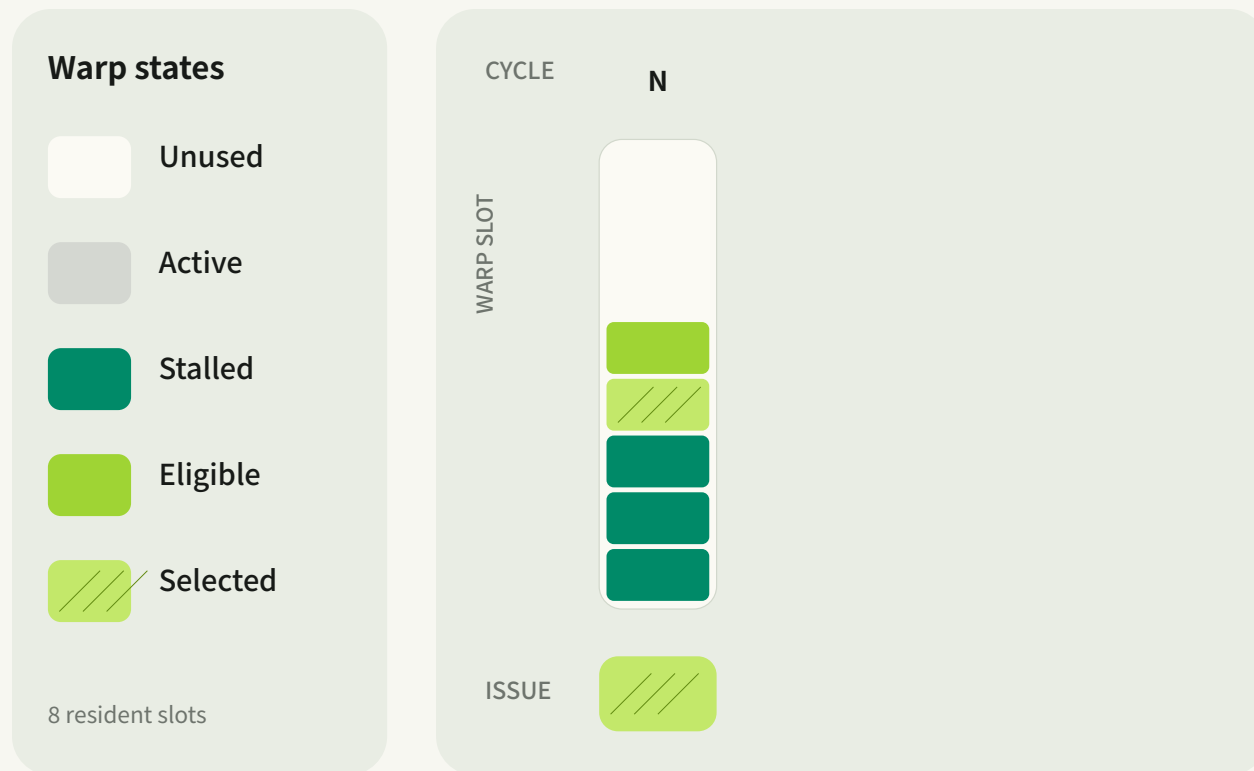
CeTZ: cycle 时间线，W0:LD 发出后 consumer 等待，其他 warp 发射。

## Episode 2 — Occupancy 与 In-Flight Work

occupancy 公式、theoretical/achieved 区分、in-flight work 来源。



# Episode 2 — NCU Scheduler 统计



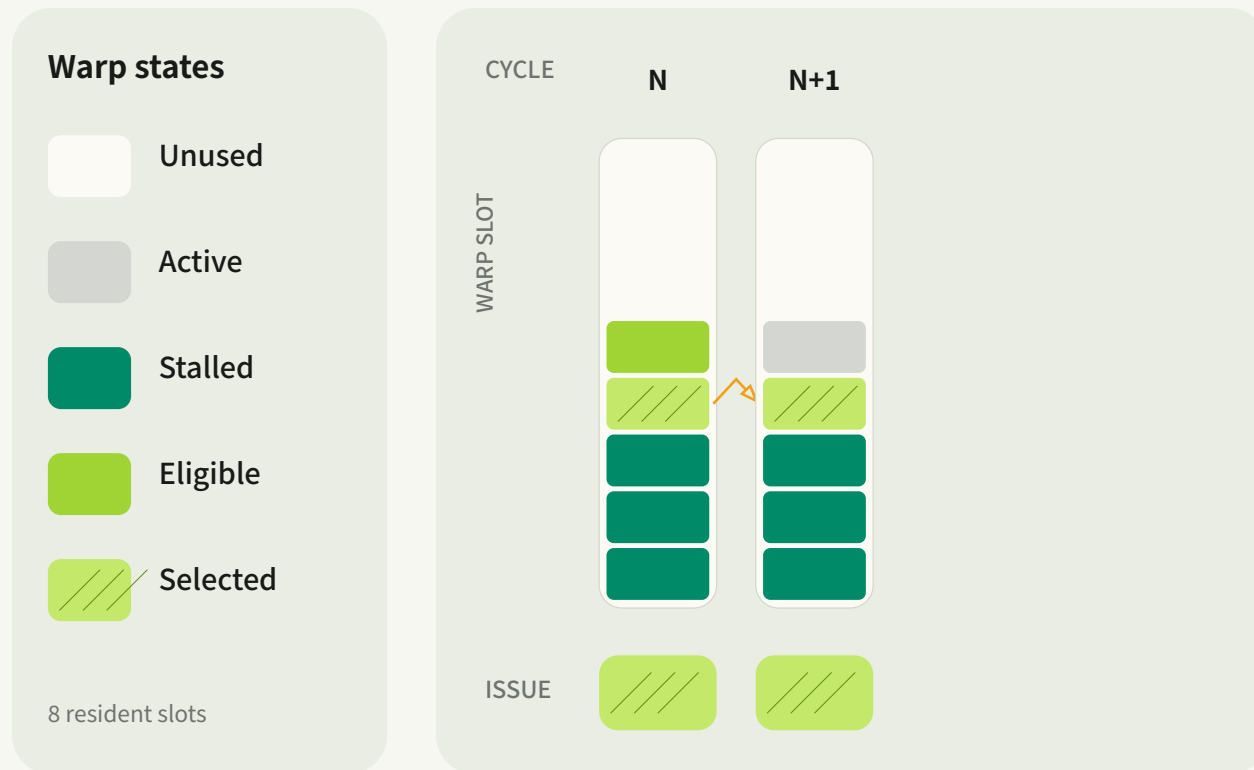
01 / N

Issue = issued:  
scheduler 选中一个 warp  
并发射。

## Derived metrics

5 active • 2 eligible • 3  
stalled • 0 bubble

# Episode 2 — NCU Scheduler 统计



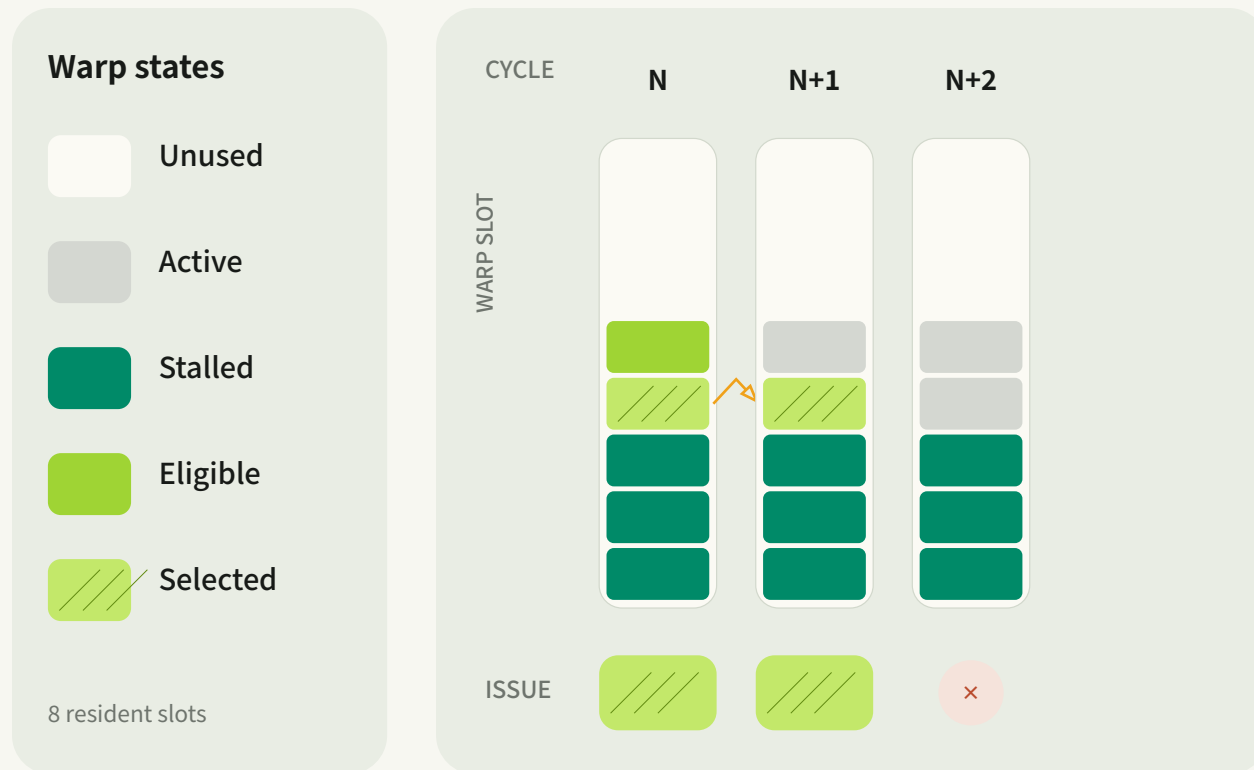
02 / N+1

Issue = issued:  
scheduler 选中一个 warp  
并发射。

## Derived metrics

10 active • 3 eligible • 7  
stalled • 0 bubble

# Episode 2 — NCU Scheduler 统计



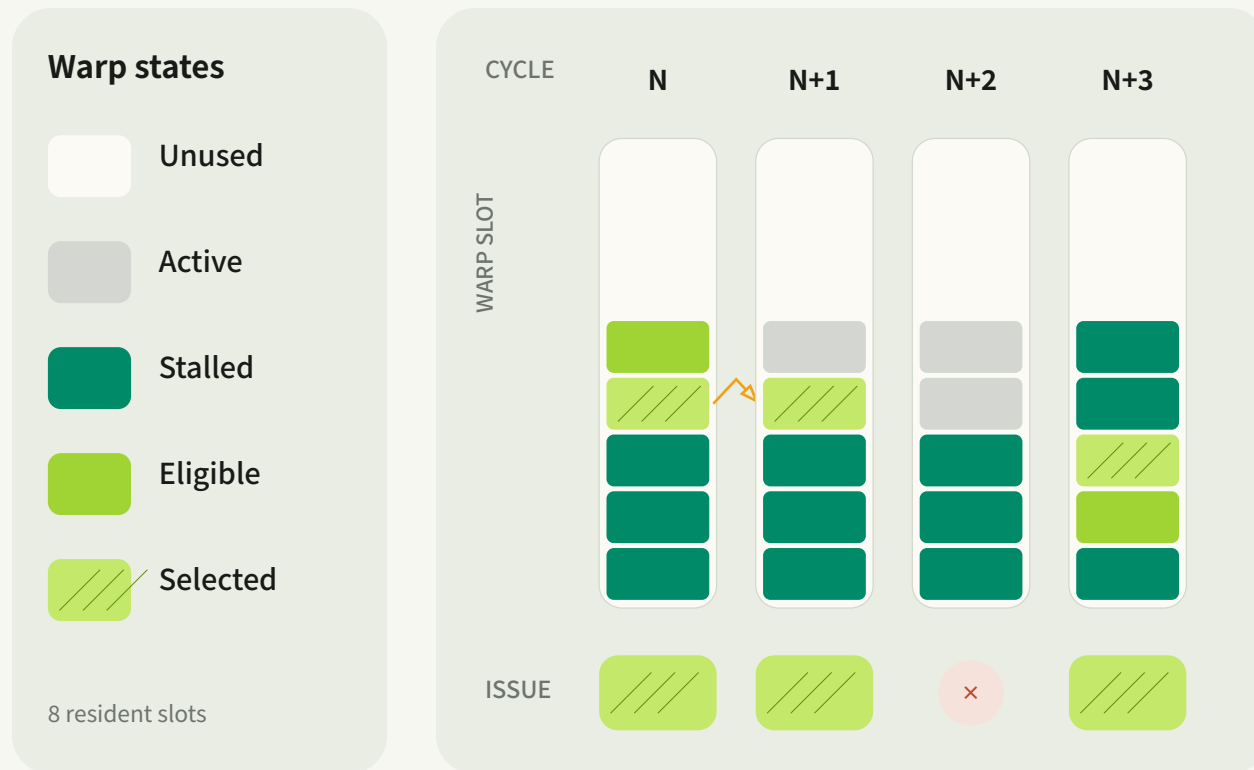
03 / N+2

Issue = bubble: 没有 eligible warp, 发射槽空置。

## Derived metrics

15 active • 3 eligible • 12 stalled • 1 bubble

# Episode 2 — NCU Scheduler 统计



04 / N+3

Issue = issued:  
scheduler 选中一个 warp  
并发射。

## Derived metrics

20 active • 5 eligible • 15  
stalled • 1 bubble

## Episode 3 — 按 Reuse Scope 看 Memory Hierarchy

五层 hierarchy 表：scope / controller / 课程例子。

## Episode 3 — 为什么 Cache 不替代 Shared Memory

Cache: 硬件管理、机会性、可能被替换/竞争。

Shared: 显式寻址、block-scoped、需要 load/sync/layout 设计。

## Episode 3 — Tiled GEMM 与两个 Barrier

barrier 1: tile 装载完整； barrier 2: 安全覆盖 shared buffer。

code-block: V2 tiled GEMM 主循环（含两个 barrier）。

## Episode 3 — 第一次 Roofline 右移

CeTZ: Roofline 上 naive → tiled 的 AI 右移。

shared tiling 减少 logical global loads; 实测 bytes/FLOP 决定右移幅度。



## Episode 4 — 片上资源与 Occupancy Trade-off

larger tile → more reuse → faster; larger tile → fewer resident warps → slower。

occupancy 是 trade-off 的一侧，不是最终目标。

## Episode 5 — Banked Shared SRAM

高 aggregate bandwidth，而不是  
32 个完全独立任意地址端口。

CeTZ: 32 banks 环形映射；同一 bank 不同地址 = conflict，同地址 = broadcast。

## Episode 5 — Transpose 微基准与 Padding

code-block: `__shared__ float tile[32][32];` 列访问冲突。

code-block: `__shared__ float tile[32][33];` row stride +1 打散冲突。

# Episode 6 — Register File 与 Microtiles

HBM/L2 → shared 捕获 block-level reuse; shared → register 捕获 thread-level reuse。

code-block: V3/V4 outer-product 主循环 (reg\_a/reg\_b/acc)。

## Episode 7 — L2 Locality 与 Block Ordering (附录)

physical blockIdx → logical tile 的映射；只谈概率，不保证顺序。

## SECTION

# Part 3 — reduce

## OBJECTIVE

在一个有跨线程归约依赖的 workload 上复用前两部分的推理链。

## LISTEN FOR

竞争应该发生在哪一层？每往下推一层能省多少？

## reduce: 问题

给定长度  $N$  的数组，求和：

$$\text{out} = \sum_{i=0}^{N-1} x_i$$

实测环境：NVIDIA A100 80GB PCIe • CUDA 12.9 •  $N = 2^{26}$  (256 MB, float32)

A100 measured

# V1: 每个线程一次 atomicAdd

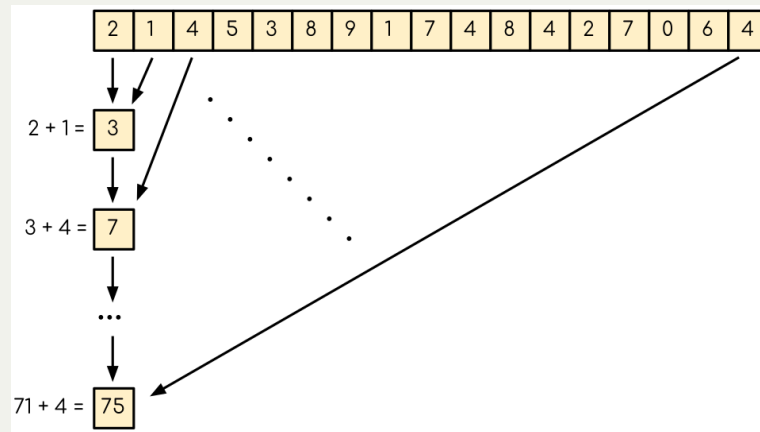
```
__global__ void reduce_v1(  
    const float* x, float* out, int n)  
{  
    int i = blockIdx.x * blockDim.x  
        + threadIdx.x;  
    if (i < n)  
        atomicAdd(out, x[i]);  
}
```

CUDA

最直接的并行化： $N$  个线程，每个线程把自己那个数加到**同一个地址**上。正确性由硬件保证 —— `atomicAdd` 的读-改-写不会被打断。

实测 197 ms, 1.4 GB/s

慢了三个数量级。





## V1 为什么这么慢

原子加**不是在线程里做的**。SM 把「地址 + 操作数 + 操作码」打包成请求，送到该地址所属的 **L2 slice**，由 L2 里的 ALU 完成读-改-写。

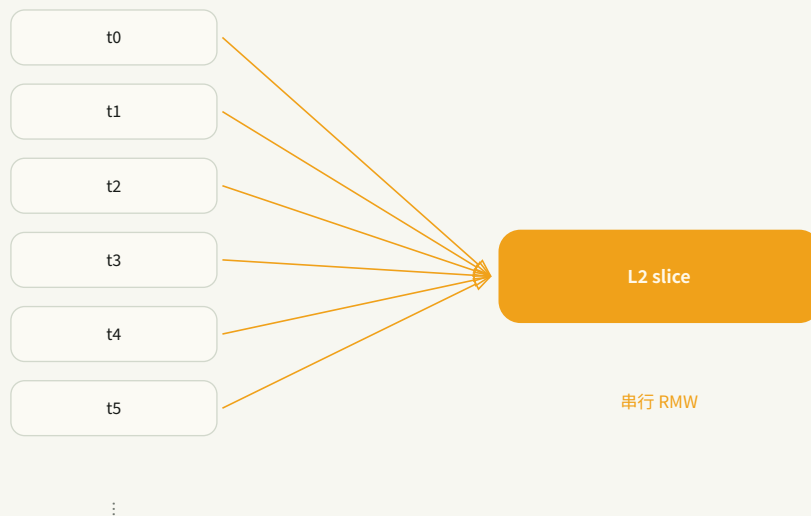
原子性来自**只有一个执行点**，不需要锁 —— 但同一个地址的请求也因此必须**排队串行**。

### 瓶颈在哪

不是带宽，也不是延迟，而是**单个 L2 slice 对同一地址的 RMW 吞吐**。

```
l1tex__t_requests___op_red = 131,072  
lts__t_sectors_op_red      = 7,560,448
```

每个请求在 L2 上放大 58 倍，全部堆在一个地址上排队 **NCU model**



# V2: Shared Memory 树形归约

CUDA

```

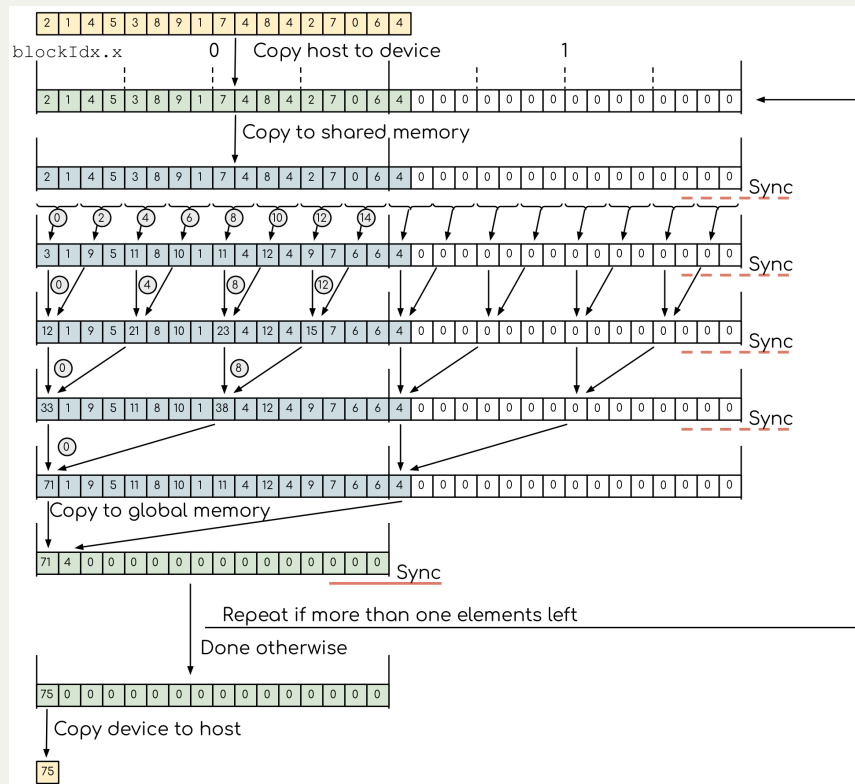
__shared__ float s[BLOCK];
int tid = threadIdx.x;
int i = blockIdx.x * blockDim.x + tid;
s[tid] = (i < n) ? x[i] : 0.0f;
__syncthreads();

for (int stride = 1;
     stride < blockDim.x; stride *= 2) {
    if (tid % (2 * stride) == 0)
        s[tid] += s[tid + stride];
    __syncthreads();
}
if (tid == 0) atomicAdd(out, s[0]);

```

每个 block 先在**自己的 smem** 里归约成 1 个数，再 atomic 一次。原子操作从  $N$  次降到  $N/256$  次。

1.67 ms, 160 GB/s — 比 V1 快 **118 倍**



## V2 的问题: 活跃线程是散开的

看图里圈出的线程号: 0, 2, 4, 6, ...

条件是  $\text{tid} \% (2 * \text{stride}) = 0$ , 活跃线程**均匀散布在整个 block 里**。

一个 warp 的 32 个 lane 中, 第一轮只有 16 个干活, 第二轮 8 个……但**整个 warp 都必须被调度**, 没干活的 lane 只是被 predicate 掉。

代价在指令条数上

V2 执行了 **1.55 亿** 条 warp 指令。

stride = 1 时的一个 warp



16/32 活跃, 但整个 warp 都在跑

V4 的排布



活跃线程连续 → 整 warp 可退出

# V3: 让活跃线程连续

```
for (int stride = 1;
     stride < blockDim.x; stride *= 2) {
    int idx = 2 * stride * tid;
    if (idx < blockDim.x)
        s[idx] += s[idx + stride];
    __syncthreads();
}
```

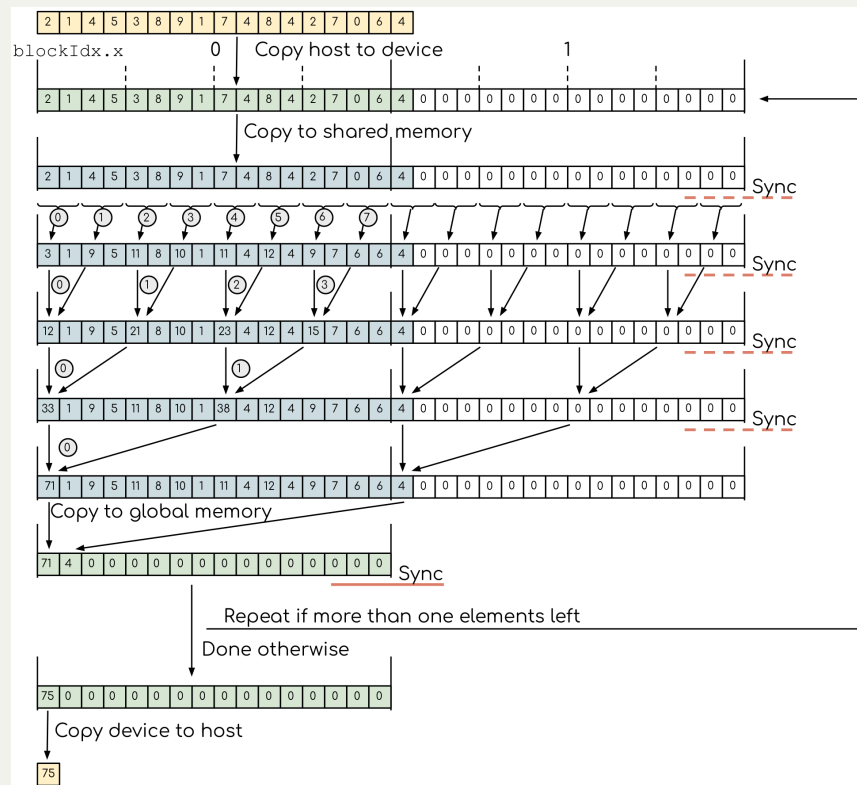
CUDA

改用 `idx = 2*stride*tid`：干活的是 **tid 0,1,2,3...**，连续。

图里圈出的线程号变成了 `0, 1, 2, 3, ...` —— 后面的 warp 可以**整个退出**。

指令数从 1.55 亿降到 **6167 万** ( $\div 2.5$ )。

0.87 ms, 308 GB/s —— 比 V2 快 **1.9 倍**



## 但 V3 引入了 bank conflict

`s[idx]`, `idx = 2*stride*tid` —— 访问是**跨步的**。

smem 有 32 个 bank, 跨步 2 → 一半 lane 撞同一 bank, 跨步 4 → 1/4……**一次请求要拆成多个 wavefront**。

|           | bank conflict    | wavefront/请求 |
|-----------|------------------|--------------|
| V2        | 40 093           | 1.01         |
| <b>V3</b> | <b>6 934 667</b> | <b>1.49</b>  |
| V4        | 34 354           | 1.01         |

### 一个值得注意的细节

V3 的 bank conflict 比 V2 **多了 170 倍**, 但它仍然**更快**。

因为 V2 的瓶颈是**指令条数** (1.55 亿), V3 用  $2.5\times$  的指令削减, 盖过了 conflict 带来的  $1.49\times$  wavefront 放大。

V3 比 V4 多出的 699 万个 wavefront, 与它的 693 万次 bank conflict 几乎完全对应。

# V4: Sequential Addressing

```
for (int stride = blockDim.x / 2;
    stride > 0; stride >>= 1) {
    if (tid < stride)
        s[tid] += s[tid + stride];
    __syncthreads();
}
```

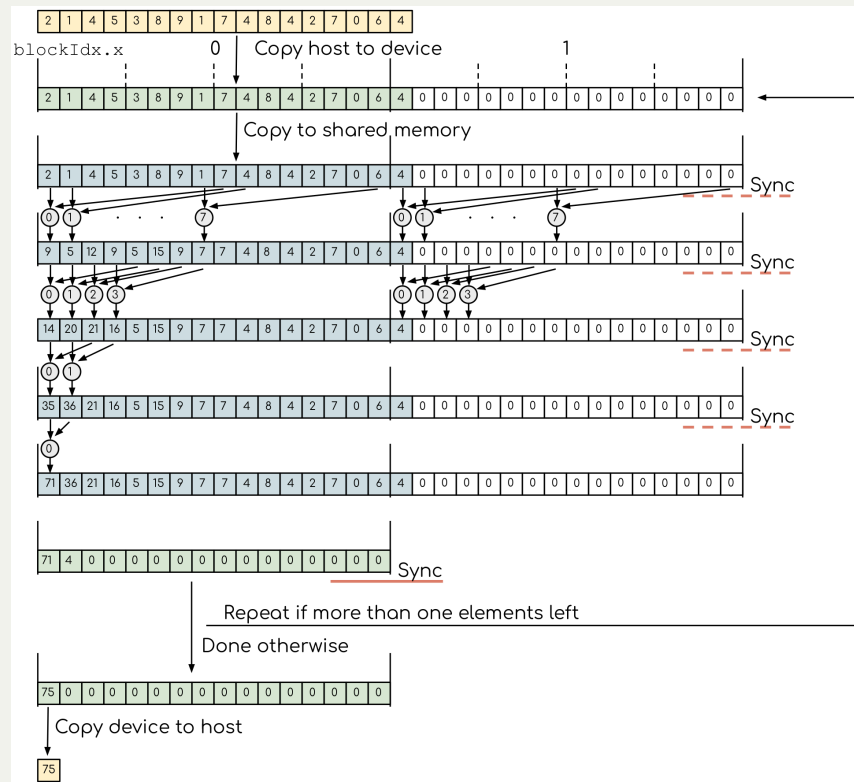
CUDA

stride 从大到小折半，活跃线程始终是前 stride 个。

两个好处同时拿到：

- 活跃线程连续 → 无 divergence
- `s[tid]` 连续访问 → 无 bank conflict

bank conflict: 693 万 → 3.4 万 0.78 ms, 344 GB/s



## V5: 让每个线程多干活

前面几版都是一个元素一个线程， $2^{26}$  个元素要开 26 万个 block。而且载入后第一轮就有一半线程闲置。

改成**固定网格 + grid-stride 循环**：每个线程先串行累加多个元素。

```
float sum = 0.0f;
for (int i = blockIdx.x*blockDim.x + tid;
     i < n; i += blockDim.x*blockDim.x)
    sum += x[i];
s[tid] = sum;
__syncthreads();
// ... 之后照常折半归约
```

CUDA

这一步是全场最大的提升

0.78 ms → **0.25 ms**

344 GB/s → **1074 GB/s**

原因：

- 每线程有**多个独立的加法在飞** → 访存并行度 (MLP) 上来了
- block 数固定为  $SM \times 16$ ，**不再有启动开销和尾部效应**
- 归约的树只需要做**一次**，而不是 26 万次

指令数：5276 万 → **560 万** ( $\div 9.4$ )

## V6: Warp Shuffle

折半归约到 `stride ≤ 16` 时，参与的线程**全在一个 warp 内**。

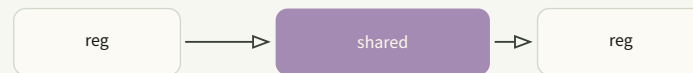
warp 内的 32 个 lane 可以**直接交换寄存器**，根本不需要经过 shared memory。

```
__device__ float warp_reduce_sum(float v)
{
    #pragma unroll
    for (int off = 16; off > 0; off >>= 1)
        v += __shfl_xor_sync(0xffffffff,
                             v, off);
    return v;
}
```

CUDA

SASS: `SHFL.BFLY PT, R3, R4, 0x10, 0x1f`

smem



写 → 同步 → 读

shuffle



一条指令，几个 cycle



# Shuffle 指令族

```
T __shfl_sync (unsigned mask, T var, int srcLane, int width = 32);
T __shfl_up_sync (unsigned mask, T var, unsigned delta, int width = 32);
T __shfl_down_sync(unsigned mask, T var, unsigned delta, int width = 32);
T __shfl_xor_sync (unsigned mask, T var, int laneMask, int width = 32);
```

CUDA

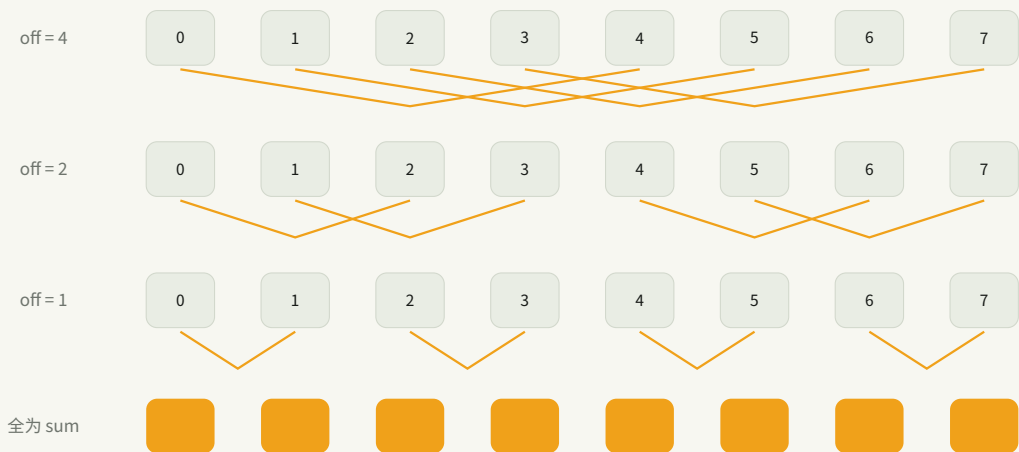
语义统一：每个 lane 交出自己的 `var`，再按规则取回某个 lane 的值。

CUDA contract

| 指令                            | lane $i$ 拿到谁的值                   | 典型用途               |
|-------------------------------|----------------------------------|--------------------|
| <code>__shfl_sync</code>      | lane <code>srcLane</code> (任意指定) | 广播、置换              |
| <code>__shfl_up_sync</code>   | lane $i - \text{delta}$          | prefix sum、stencil |
| <code>__shfl_down_sync</code> | lane $i + \text{delta}$          | 归约（结果在 lane 0）     |
| <code>__shfl_xor_sync</code>  | lane $i \oplus \text{laneMask}$  | 蝶形归约（全 lane 得结果）   |

越界的 lane（如 up 时  $i < \text{delta}$ ）保留自己原来的值，不是 0

# 蝶形归约的过程



**xor (蝶形)**：所有 lane 都拿到结果。后面每个 lane 都要用这个值时（softmax 除以 sum）省一次广播。

**down (折半)**：只有 lane 0 有效。指令条数完全相同，选哪个取决于结果要出现在几个 lane 上。

# mask 与 width

`mask`：哪些 lane 参与这条指令

- 通常写 `0xffffffff` (全 32 个 lane)
- 但在**分支内部**，若有 lane 没执行到这条指令，写全 1 是**未定义行为**
- 此时应当用 `__activemask()`，或精确的常量掩码

## Volta 之后

independent thread scheduling 引入后，warp 内不再保证隐式锁步，旧的无 `__sync` 版本 `__shfl` **已被移除**。忘记 `mask` 是一个很隐蔽、且只在特定 GPU 上复现的 bug。

`width`：把 warp 切成若干子段

- 默认 32；设为 8 则 warp 分成 4 段，shuffle **只在段内进行**
- 适合「每 8 个 lane 处理一行」这类布局

两个参数都属于 CUDA 编程契约，不随架构变化 —— 但它们保证的是正确性，不是性能。

# V6 完整实现: 两级归约

CUDA

```

__global__ void reduce_v6(const float* x, float* out, int n) {
    float sum = 0.0f;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
         i += gridDim.x * blockDim.x)
        sum += x[i];

    sum = warp_reduce_sum(sum);           // ① warp 内: 纯 shuffle

    __shared__ float warp_sum[32];       // 一个 block 最多 32 个 warp
    int lane = threadIdx.x & 31, wid = threadIdx.x >> 5;
    if (lane == 0) warp_sum[wid] = sum;   // ② 每 warp 写 1 个值
    __syncthreads();                     // 整个 kernel 只同步一次

    if (wid == 0) {                       // ③ 第一个 warp 再归约一次
        sum = (lane < blockDim.x / 32) ? warp_sum[lane] : 0.0f;
        sum = warp_reduce_sum(sum);
        if (lane == 0) atomicAdd(out, sum);
    }
}

```

smem 用量 4 KB → 128 B; `__syncthreads()` 8 次 → 1 次

## V7: 再加上向量化访存

到 V6 为止，瓶颈已经完全回到**读输入**上了。一次  
`float4` 读 16 字节，**指令条数减少到 1/4**。

```
const float4* x4 = (const float4*)x;
for (int i = ..., i < n4; i += stride) {
    float4 v = x4[i];
    sum += v.x + v.y + v.z + v.w;
}
```

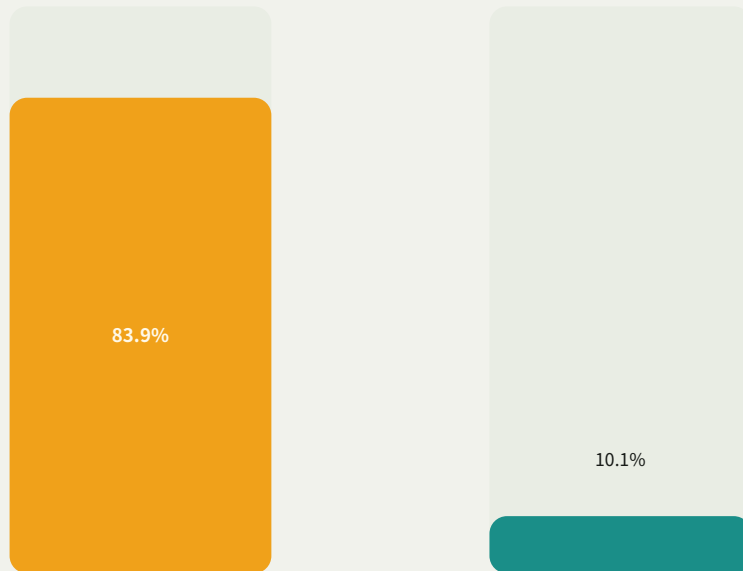
CUDA

指令数 460 万 → 185 万

0.17 ms, 1572 GB/s

ncu 报告: **83.9% of DRAM peak**

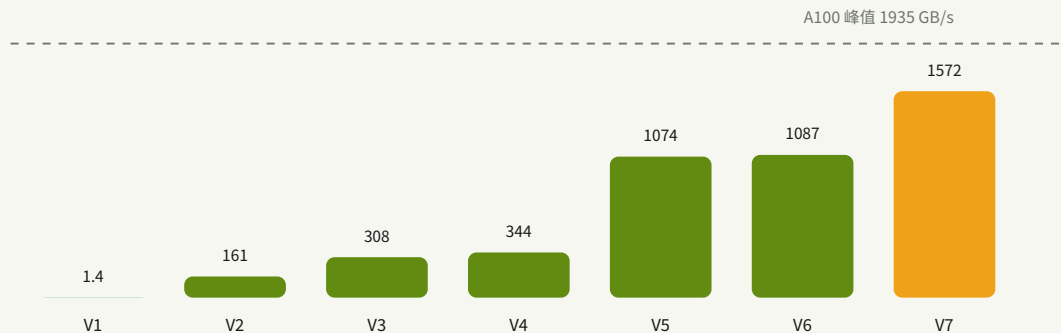
Compute throughput 仅 10.1%



# 实测结果

| 版本                 | 时间             | 带宽               | vs V1        | warp 指令 | 结果正确 |
|--------------------|----------------|------------------|--------------|---------|------|
| V1 global atomic   | 197.4 ms       | 1.4 GB/s         | 1×           | —       | ✗    |
| V2 smem, 散开        | 1.67 ms        | 161 GB/s         | 118×         | 1.55 亿  | ✓    |
| V3 连续线程            | 0.87 ms        | 308 GB/s         | 227×         | 6167 万  | ✓    |
| V4 sequential      | 0.78 ms        | 344 GB/s         | 253×         | 5276 万  | ✓    |
| V5 grid-stride     | 0.25 ms        | 1074 GB/s        | 790×         | 560 万   | ✓    |
| V6 warp shuffle    | 0.247 ms       | 1087 GB/s        | 799×         | 460 万   | ✓    |
| <b>V7 + float4</b> | <b>0.17 ms</b> | <b>1572 GB/s</b> | <b>1156×</b> | 185 万   | ✓    |

A100 80GB PCIe •  $N = 2^{26}$  • 20 次平均 • 排序在  $2^{22} \sim 2^{28}$  范围内稳定 A100 measured



## 小结

1. **reduce 的上限是把输入读一遍的时间**，所有优化都在逼近这条线
2. **V1 的问题不是带宽，是竞争**：原子操作在 L2 上串行，而且 float32 累加到  $2^{24}$  就**停止增长**
3. **V2→V3 的收益来自指令数，不是 divergence 本身**：V3 的 bank conflict 反而多了 170 倍，但指令少了 2.5 倍，所以更快
4. **V4 同时消掉两者；V5 的 grid-stride 是全场最大的一步** ( $3.1\times$ )
5. **V6 用 shuffle 省掉同步和 smem**，为后续融合 (softmax / norm) 留出空间

**竞争往下推一层，代价小一个数量级**

# NCU 详细走查

解释指标：sectors、Long/Short Scoreboard、excessive wavefronts、% peak。

code-block: 一条典型 ncu 命令行 (-section 列表)。



## SECTION

# Act III — Rebuild

## OBJECTIVE

按 V0 → V6 重建 kernel，用 correctness harness 与 NCU 验证瓶颈迁移。

## LISTEN FOR

每一步在消除哪个瓶颈？下一个瓶颈是什么？

## 通用 Kernel 设计流程

Workload: 找出复用、判断 scope。

Hardware: 放到匹配的 memory level、  
按 warp 推 lane 映射。

Measurement: 检查 banks/registers、  
保留 in-flight work、NCU 验证。

## V0 — Naive GEMM

AI\_code  $\approx 0.25$ ; 预期 NCU: global loads 多、Long Scoreboard 可能高、FP32 利用率低。

code-block: `gemm_v00_naive` 内核。

# V1 — Warp-Friendly Mapping

code-block: 错误 mapping ( $\text{threadIdx.x} \rightarrow \text{row}$ )。

code-block: 修正 mapping ( $\text{threadIdx.x} \rightarrow \text{col}$ )。

## V2 — Shared-Memory Block Tiling

预期 NCU：DRAM/L2 bytes 下降、  
AI\_HBM 上升、shared/barrier 开始可见。

code-block: V2 tiled 主循环。

## V3 — 1D Register Tiling

shared tiling 减 global traffic; register  
tiling 开始减 shared traffic。

code-block: V3 1D register 主循环。

## V4 — 2D Register Outer Product

提高 register reuse 的 ownership 不一定天然适合 shared banks。

code-block: V4 outer-product 主循环。

## V5 — Vectorized Global Loads

Vector width 与 warp-level coalescing 是不同问题。

code-block: V5 load\_A\_tile / load\_B\_tile 的 float4 路径。



## V6 — Bank-Aware Lane Ownership

Thread tile 同时规定 register reuse、shared banks 与 global coalescing。

code-block: V6 striped 主循环 + C store。

# NCU 验证 workflow

顺序: Duration → SOL → Memory →  
Scheduler → Warp → Source →  
Occupancy。

code-block: 一条完整 ncu 命令行 (-section ... -o ...)。

## 结果与验收标准

表格: V0 0.43 → V6 10.97 TFLOP/s; cuBLAS pedantic 14.41; 76%。



SECTION

# Summary



# Takeaways

三条 takeaway 收束 + 瓶颈迁移图。

## 后续课程接口

当前 load→wait→compute→wait→load；后续如何隐藏剩余数据移动。



SECTION

# Appendix



# 裁剪与延后主题

60 分钟裁剪清单 + 延后主题（双缓冲、cp.async、TMA、Tensor Core）。



# Hierarchical Roofline (Multi-Boundary)

不同 memory level 用不同 denominator; shared roof 是 pattern-specific 课程模型。

CeTZ: 多边界 roofline 的层级示意。